

Respuestas a Preguntas de Entrevista para Arquitectos de Software

1. What's your approach to multi-tenant data models?

Existen tres estrategias fundamentales para modelar multi-tenancy, y la elección depende del balance entre aislamiento, costo y complejidad operacional.

Database-per-tenant significa que cada inquilino tiene su propia base de datos física. El aislamiento es total: no hay riesgo de data leakage, los backups y restores son independientes, y se puede escalar cada tenant de forma individual. El problema es el costo operacional — si tienes 10,000 tenants, tienes 10,000 bases de datos que necesitan migraciones, monitoreo y backups. Las conexiones se multiplican y necesitas un mecanismo de routing (típicamente un catálogo central que mapea `tenant_id` → `connection_string`).

Schema-per-tenant es un punto intermedio donde todos los tenants comparten la misma instancia de base de datos, pero cada uno tiene su propio schema (en PostgreSQL, por ejemplo, cada tenant tiene su schema con sus propias tablas). La ventaja es que reduces el overhead operacional porque es una sola instancia, pero mantienes aislamiento lógico. Las migraciones se complican porque debes iterar sobre todos los schemas. En PostgreSQL esto funciona bien hasta ~1,000 schemas; más allá, el catálogo del sistema se degrada.

Shared-table con discriminator column es la estrategia más común para SaaS a escala. Todas las filas de todos los tenants viven en las mismas tablas, diferenciadas por una columna `tenant_id`. Es la más eficiente en recursos y la más simple operacionalmente (una migración aplica a todos), pero el riesgo de data leakage es real. Necesitas Row-Level Security (RLS) en PostgreSQL, o un filtro a nivel de aplicación que nunca pueda omitirse (típicamente un interceptor o middleware que inyecta el `WHERE tenant_id = ?` en cada query).

Cómo lo implementaría en la práctica: Para la mayoría de los productos SaaS, usaría shared-table con RLS de PostgreSQL. La política se define así:

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;  
CREATE POLICY tenant_isolation ON orders  
  USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

Y al inicio de cada request, se ejecuta `SET app.current_tenant = '<uuid>'`. Esto garantiza que incluso si un desarrollador olvida el filtro en un query, la base de datos rechaza filas de otro tenant.

Follow-ups anticipados:

¿Cómo manejas tenants con volúmenes masivamente diferentes? — Uso un modelo híbrido: los tenants enterprise con alto volumen se mueven a database-per-tenant (lo que se llama "silo model" en AWS), mientras los tenants small/medium comparten tablas ("pool model"). El routing entre ambos se maneja en un servicio de tenant resolution que consulta un registro central.

¿Cómo haces migraciones en shared-table? — Online schema migration con herramientas como

`pg_repack` o `gh-ost` (para MySQL) para evitar locks. Nunca hago `ALTER TABLE` con lock exclusivo en producción.

¿Cómo manejas el noisy neighbor problem? — Rate limiting por tenant, connection pooling con límites por tenant (usando PgBouncer con configuración per-database), y query timeouts. Si un tenant ejecuta queries pesados, no debe degradar a los demás.

2. What patterns reduce the risk of deadlocks in transactional systems?

Un deadlock ocurre cuando dos o más transacciones se bloquean mutuamente esperando recursos que la otra tiene. La prevención se basa en principios que eliminan las condiciones necesarias para que ocurra el ciclo.

Ordenamiento consistente de recursos (canonical ordering): Esta es la técnica más fundamental y efectiva. Si todas las transacciones adquieren locks sobre recursos en el mismo orden predeterminado (por ejemplo, siempre por el ID de la entidad en orden ascendente), los ciclos de espera se vuelven imposibles. Si necesitas actualizar las cuentas 42 y 17, siempre lockeas la 17 primero y luego la 42, independientemente de qué operación estés haciendo. Esto requiere disciplina de equipo y code reviews estrictos.

Reducir la duración y el scope de las transacciones: Mientras más corta la transacción, menos tiempo se mantienen los locks y menos probabilidad hay de colisión. Esto significa mover cualquier operación que no necesite ser transaccional fuera de la transacción — llamadas HTTP, cálculos pesados, I/O de archivos. La transacción debería abrirse lo más tarde posible y cerrarse lo antes posible.

Optimistic Concurrency Control (OCC): En lugar de adquirir locks de escritura al inicio, lees los datos, trabajas con ellos, y al momento de escribir verificas que nadie los haya modificado (usando un campo `version` o `updated_at`). Si hay conflicto, haces retry. Esto es ideal cuando la contención es baja (la mayoría de los intentos no colisionan). En JPA/Hibernate se implementa con `@Version`:

```
@Entity
public class Account {
    @Version
    private Long version;
    private BigDecimal balance;
}
```

Si dos transacciones leen la versión 5, la primera que haga commit incrementa a 6. La segunda recibe un `OptimisticLockException` y reintenta.

Advisory locks para coordinar sin bloquear filas: En PostgreSQL puedes usar `pg_advisory_lock(hash)` para coordinar acceso a un recurso lógico sin lockear filas físicas. Esto evita que el lock manager de la base de datos se sobrecargue.

Diseño de esquema para evitar hotspots: Si muchas transacciones necesitan actualizar la misma fila (por ejemplo, un contador global), es mejor usar técnicas como "batched counter updates" o tablas de append-only con agregación periódica.

Follow-ups anticipados:

¿Cómo detectas deadlocks en producción? — PostgreSQL automáticamente detecta deadlocks y cancela una de las transacciones (configurable con `deadlock_timeout`, default 1s). Monitoreo la métrica `pg_stat_database.deadlocks` y alerto si supera un threshold. También analizo los logs con `log_lock_waits = on` para identificar qué queries compiten.

¿OCC o pessimistic locking? — OCC cuando la contención es baja y los retries son baratos. Pessimistic (SELECT FOR UPDATE) cuando la contención es alta y no puedes permitirte retries frecuentes, como en transferencias financieras donde el costo del retry incluye re-validar reglas de negocio complejas.

¿Cómo manejas deadlocks en microservicios (distributed)? — Sagas con compensación. No hay un lock manager distribuido viable a escala. En su lugar, cada servicio ejecuta su transacción local y, si falla un paso posterior, se disparan operaciones compensatorias (refund, revert, etc.).

3. How do you decide between computed views, materialized views, and real-time aggregations?

La decisión depende de tres variables: la frescura requerida de los datos, el volumen de datos, y la frecuencia de lectura versus escritura.

Computed views (vistas regulares) son simplemente queries almacenados que se ejecutan cada vez que se consultan. No almacenan datos — son un alias para un SELECT complejo. Son ideales cuando el volumen de datos es pequeño, el query es rápido (<100ms), y siempre necesitas datos 100% frescos. El problema es que si el query subyacente hace JOINS pesados o agregaciones sobre millones de filas, cada lectura paga ese costo.

Materialized views son el resultado pre-computado de un query almacenado físicamente en disco. Se refrescan bajo demanda (`REFRESH MATERIALIZED VIEW`). Son ideales cuando puedes tolerar datos ligeramente desactualizados (staleness de minutos u horas), las lecturas son muy frecuentes, y el cómputo subyacente es caro. El ejemplo clásico: un dashboard de analytics que muestra métricas del último mes y se refresca cada 15 minutos. En PostgreSQL, `REFRESH MATERIALIZED VIEW CONCURRENTLY` permite refrescar sin bloquear lecturas, pero requiere un índice unique.

Real-time aggregations (CQRS + event streaming) son necesarias cuando requieres datos frescos en tiempo real sobre volúmenes masivos. Aquí usas una arquitectura donde cada evento (compra, click, transacción) se publica a un stream (Kafka, Kinesis), un consumidor actualiza una vista pre-agregada en una store optimizada para lectura (Redis, Elasticsearch, una tabla de resúmenes en la DB), y las lecturas van contra esa store. El costo es complejidad arquitectónica significativa.

Mi framework de decisión: Pregunto primero "¿cuánto staleness tolera el negocio?" Si la respuesta es "cero, siempre actual", evalúo el costo del computed view. Si es aceptable (<200ms p99), uso computed view. Si no, voy a real-time aggregation. Si el negocio tolera 5-60 minutos de retraso, materialized view con refresh cron. Si tolera horas o más, materialized view con refresh nocturno o un pipeline batch (dbt, Airflow).

Follow-ups anticipados:

¿Qué haces cuando la materialized view tarda mucho en refrescar? — Refresh incremental. En lugar de recalcular todo, mantengo una tabla de "cambios desde el último refresh" y solo proceso los deltas. Esto se puede hacer con un trigger que llena una tabla de changelog, o con CDC (Change Data Capture) usando Debezium.

¿Dónde encaja dbt en esto? — dbt es excelente para materialized views gestionadas como código. Define el query, la estrategia de materialización (table, incremental, ephemeral), y lo ejecuta en un schedule. Es el estándar para data transformation en el data warehouse.

¿Y para métricas como "usuarios online ahora"? — Eso es definitivamente real-time. Usaría Redis con un sorted set donde el score es el timestamp del último heartbeat, y un query `ZRANGEBYSCORE` para contar usuarios activos en los últimos 5 minutos.

4. How do you design a data-retention and archival strategy?

La retención de datos no es solo un problema técnico — es un problema de compliance, costo y rendimiento que requiere alineación con legal, negocio y operaciones.

Clasificación de datos por tier: Primero clasifico los datos en categorías con políticas distintas. Los datos transaccionales calientes (últimos 3-6 meses) viven en la base de datos principal con acceso rápido. Los datos tibios (6 meses a 2-3 años) se mueven a almacenamiento más barato pero aún queryable (por ejemplo, tablas particionadas con particiones antiguas en tablespaces de almacenamiento económico, o un data warehouse como Redshift/BigQuery). Los datos fríos (>2-3 años) van a object storage (S3 con Glacier) en formato comprimido como Parquet.

Particionamiento como enabler: PostgreSQL permite particionar tablas por rango de fechas. Cada partición (por ejemplo, mensual) es una tabla física separada. Archivar datos viejos es tan simple como `ALTER TABLE DETACH PARTITION` — la partición se convierte en una tabla independiente que puedes exportar a S3 y luego `DROP`. Esto es órdenes de magnitud más rápido que `DELETE FROM` sobre millones de filas, que genera WAL masivo y bloquea la tabla.

Cumplimiento regulatorio: Diferentes regulaciones imponen diferentes tiempos de retención. GDPR exige que puedas borrar datos personales a solicitud (right to erasure), lo que complica el archival porque necesitas poder encontrar y eliminar datos de un usuario específico en todos los tiers, incluyendo backups. SOX requiere retención mínima de 7 años para datos financieros. PCI-DSS tiene reglas específicas sobre datos de tarjetas. La estrategia de archival debe documentar qué regulación aplica a cada tipo de dato y el período mínimo y máximo de retención.

Soft delete vs hard delete: Para datos que eventualmente se borrarán, uso soft delete (`deleted_at` timestamp) inicialmente. Un job nocturno mueve los soft-deleted después de N días a la tabla de archivo, y otro job purga los archivos después de que expira el período de retención. Esto da una ventana para recuperación y audit.

Follow-ups anticipados:

¿Cómo manejas GDPR right-to-erasure con backups? — Los backups se consideran "datos de procesamiento limitado". La práctica aceptada es que no modificas backups existentes (eso rompería su integridad), pero si restauras un backup, el proceso de restauración debe aplicar las solicitudes de erasure pendientes. Mantengo un registro de erasure requests con su timestamp, y

cualquier restore ejecuta ese registro como post-step.

¿Cómo mantienes queries rápidos cuando la tabla crece? — Particionamiento + índices parciales. El query planner de PostgreSQL hace partition pruning automático si la cláusula WHERE incluye la partition key. Además, puedo crear índices solo en las particiones calientes.

5. How do you design authentication and authorization across distributed systems?

Este es uno de los problemas más críticos en arquitecturas de microservicios porque cada servicio necesita saber quién es el usuario y qué puede hacer, sin acoplar todos los servicios a un solo sistema de identidad.

Autenticación centralizada con tokens distribuidos: Uso un Identity Provider (IdP) centralizado — ya sea un servicio propio con OAuth2/OIDC o un proveedor como Keycloak, Auth0, o AWS Cognito. El flujo es: el cliente se autentica contra el IdP y recibe un JWT (JSON Web Token). Ese JWT viaja en el header `Authorization: Bearer <token>` en cada request. Los servicios downstream validan el JWT sin contactar al IdP, porque el JWT está firmado (RS256/ES256) y el servicio tiene la public key del IdP (obtenida del endpoint `.well-known/jwks.json`).

Claims en el JWT vs lookup externo: El JWT debe contener claims mínimos pero suficientes: `sub` (user ID), `tenant_id`, roles, y `permissions` básicos. Evito poner demasiada información en el JWT porque (a) aumenta el tamaño del token en cada request, (b) los claims son inmutables hasta que el token expire, así que un cambio de permisos no se refleja inmediatamente. Para permisos granulares o dinámicos, el servicio consulta un servicio de autorización (como OpenFGA o un servicio propio) usando el `sub` del JWT.

Propagación de identidad entre servicios: Cuando el Servicio A llama al Servicio B internamente, propaga el JWT original del usuario (no crea uno nuevo). Esto mantiene la cadena de identidad y permite que el Servicio B aplique sus propias políticas de autorización basándose en quién es el usuario original. Para comunicación asíncrona (a través de colas), el JWT se incluye como atributo del mensaje, pero con cuidado: el token puede expirar antes de que el mensaje se procese. La solución es usar un service account token para la comunicación async y propagar el `user_id` como metadata.

Authorization patterns: El patrón que prefiero es Policy-Based Access Control (PBAC) usando Open Policy Agent (OPA) o un sidecar equivalente. Cada servicio consulta su policy engine local: "¿Puede el usuario X hacer la operación Y sobre el recurso Z?" La policy se define en Rego (para OPA) o en un DSL similar, y se distribuye a todos los servicios como archivos de configuración.

API Gateway como enforcement point: El gateway es el primer punto de validación. Verifica que el JWT sea válido, no haya expirado, y opcionalmente aplica políticas gruesas (rate limiting, IP allowlisting). Pero la autorización granular (¿puede este usuario editar *este* recurso específico?) debe ocurrir en el servicio que posee el recurso, porque solo ese servicio tiene el contexto completo.

Follow-ups anticipados:

¿Cómo manejas la revocación de tokens? — Los JWTs son stateless, así que no se pueden revocar individualmente sin perder esa propiedad. Uso TTL cortos (15 minutos) con refresh tokens de vida

larga almacenados en la base de datos del IdP. Si necesito revocar inmediatamente, mantengo una blacklist de tokens en Redis con TTL igual al del token. Cada servicio consulta Redis antes de aceptar el token. Esto introduce un punto de fallo (Redis), pero es un tradeoff aceptable para la seguridad.

¿Qué haces con service-to-service auth? — mTLS para identidad del servicio (cada servicio tiene su propio certificado X.509), combinado con SPIFFE/SPIRE para gestión automática de identidades de servicio. El JWT del usuario viaja como header adicional para mantener la identidad del usuario original.

6. What's your process for securing internal service-to-service communication?

La comunicación interna entre servicios no debe asumirse segura solo porque está dentro de una red privada. El modelo zero-trust asume que la red interna puede estar comprometida.

mTLS (Mutual TLS): Cada servicio tiene un certificado TLS cliente y verifica el certificado del servicio al que se conecta. Esto garantiza identidad bidireccional y encriptación en tránsito. En la práctica, implementar mTLS manualmente es operacionalmente costoso (gestión de certificados, rotación, revocación), así que uso un service mesh como Istio o Linkerd que abstrae toda esa complejidad. El sidecar proxy (Envoy en Istio) maneja mTLS automáticamente — los servicios hablan HTTP plano al sidecar, y el sidecar se encarga de TLS con el otro sidecar.

SPIFFE/SPIRE para identidad de servicio: SPIFFE define un estándar para identidades de servicio (SPIFFE IDs, como `spiffe://cluster.local/ns/payments/sa/payment-service`). SPIRE es la implementación que automatiza la emisión y rotación de certificados X.509 de vida corta (horas, no años). Esto elimina el problema de gestión de certificados de larga duración.

Políticas de red (Network Policies): En Kubernetes, las Network Policies definen qué pods pueden comunicarse entre sí. Por defecto, todos los pods pueden hablar con todos; la primera medida es aplicar una política deny-all y luego abrir explícitamente las rutas necesarias. Esto es defense-in-depth — incluso si un servicio se compromete, no puede alcanzar servicios con los que no debería hablar.

Authorization a nivel de servicio: Más allá de "¿puedo llegar al servicio?", necesitas "¿puede este servicio hacer esta operación?". Esto se implementa con Istio AuthorizationPolicy o con OPA como sidecar. Por ejemplo, el servicio de notificaciones puede llamar al servicio de usuarios para obtener emails, pero no para modificar perfiles.

Follow-ups anticipados:

¿Cómo manejas la rotación de certificados sin downtime? — Los certificados se emiten con vida corta (1-12 horas) y se rotan automáticamente por SPIRE antes de que expiren. El sidecar proxy carga los nuevos certificados sin reiniciar el servicio. Esto elimina la necesidad de ventanas de mantenimiento.

¿Service mesh no agrega latencia? — Sí, típicamente 1-3ms por hop (ida y vuelta a través de dos sidecars). Para la mayoría de los servicios es despreciable. Para paths ultra-sensibles a latencia, se

puede configurar el sidecar para bypass o usar connection pooling agresivo.

7. How do you integrate observability into the architecture (logs, metrics, traces)?

La observabilidad no se agrega después — se diseña como parte fundamental de la arquitectura. El objetivo es poder responder "¿qué está pasando?" y "¿por qué falló?" sin tener que agregar logs ad-hoc y redeployar.

Los tres pilares, interconectados:

Los logs son registros textuales de eventos discretos. Deben ser estructurados (JSON, no texto libre) para ser queryables. Cada log entry debe incluir un `trace_id` para correlación con traces, un `request_id`, `tenant_id`, y el `service_name`. Uso niveles de log de forma disciplinada: ERROR para cosas que requieren acción humana, WARN para anomalías que podrían necesitar atención, INFO para eventos de negocio significativos (pedido creado, pago procesado), y DEBUG solo en desarrollo o habilitado dinámicamente para troubleshooting.

Las métricas son series temporales numéricas que permiten agregación y alertas. Sigo el método RED para servicios: Rate (requests por segundo), Errors (porcentaje de requests fallidos), Duration (latencia en percentiles p50, p95, p99). Y el método USE para infraestructura: Utilization, Saturation, Errors para CPU, memoria, disco, red. Las métricas se exponen en formato Prometheus desde cada servicio (endpoint `/metrics`) y se recolectan con Prometheus o un agente como Grafana Agent.

Los traces son la pieza que conecta todo en sistemas distribuidos. Un trace representa el recorrido completo de una request a través de múltiples servicios. Cada servicio crea un span con su duración y metadata, y todos los spans de una request se agrupan bajo un `trace_id`. Uso OpenTelemetry como estándar para instrumentación (SDK y auto-instrumentation para frameworks como Spring Boot), y exporto los traces a Jaeger, Tempo, o AWS X-Ray.

La clave es la correlación: Un log de error en el servicio de pagos contiene un `trace_id`. Puedo ir de ese log al trace completo y ver que el servicio de inventario tardó 8 segundos en responder, lo que causó un timeout. Desde el trace, puedo ir a las métricas del servicio de inventario y ver que su CPU estaba al 95%. Esa es la observabilidad real: la capacidad de navegar entre pilares.

Implementación práctica: OpenTelemetry Collector como agente unificado que recibe logs, métricas y traces, los procesa (sampling, enrichment, batching), y los exporta a los backends correspondientes. Esto desacopla la instrumentación de la infraestructura de observabilidad — puedes cambiar de Jaeger a Tempo sin tocar el código de los servicios.

Follow-ups anticipados:

¿Cómo manejas el costo de almacenamiento de traces? — Tail-based sampling: el collector guarda el trace completo solo si contiene un error o si la latencia excede un threshold. Los traces exitosos y rápidos se muestrean al 1-5%. Esto reduce el almacenamiento en ~90% sin perder visibilidad de problemas.

¿Cómo defines las alertas sin alert fatigue? — Solo alerta sobre symptoms, no causes. Alerta si el

error rate del servicio supera 1% o si la latencia p99 excede el SLO. NO alerto si la CPU está alta (eso es una cause, y podría no impactar a los usuarios). Cada alerta debe tener un runbook asociado con pasos de diagnóstico y remediación.

8. What's your strategy for secret rotation and zero-trust principles?

Los secretos (API keys, database passwords, certificates, tokens) son el vector de ataque más común en sistemas comprometidos. La estrategia debe asumir que cualquier secreto será eventualmente filtrado, y minimizar el impacto cuando eso ocurra.

Nunca almacenar secretos en código o configuración: Los secretos viven exclusivamente en un secret manager — AWS Secrets Manager, HashiCorp Vault, o GCP Secret Manager. El servicio obtiene el secreto en runtime consultando el secret manager. Nunca en variables de entorno hardcodedas en manifiestos de Kubernetes, nunca en archivos de configuración commiteados, nunca en CI/CD logs.

Rotación automática: Vault o Secrets Manager generan credenciales dinámicas de vida corta. Por ejemplo, cuando un servicio necesita acceder a PostgreSQL, Vault genera credenciales con un TTL de 1 hora usando su database secrets engine. Cuando expiran, el servicio solicita nuevas. Si las credenciales se filtran, son inútiles después de 1 hora. Para secretos que no pueden ser dinámicos (API keys de terceros), configuro rotación automática periódica (cada 30-90 días) con Secrets Manager, que llama a una Lambda para rotar la key en el servicio externo.

Zero-trust en la práctica: Zero-trust es la filosofía de "nunca confiar, siempre verificar", aplicada a cada capa. En la red, significa que no hay "perímetro seguro" — cada comunicación entre servicios requiere autenticación y autorización (mTLS + policies). Para el acceso humano, significa que un ingeniero no tiene acceso permanente a producción; usa just-in-time access con aprobación y TTL corto (algo como Teleport o AWS SSO con session duration de 1 hora). Para los secretos, significa que el servicio demuestra su identidad (via IAM role, Kubernetes service account, o SPIFFE ID) antes de recibir el secreto — no basta con "estar en la red correcta".

Implementación con Vault: Vault es el hub central. Cada servicio se autentica con Vault usando su identidad de Kubernetes (Kubernetes auth method). Vault verifica que el pod es quien dice ser consultando la API de Kubernetes, y le entrega los secretos correspondientes a su policy. Si el pod se compromete, solo tiene acceso a los secretos de ese servicio, no a otros.

Follow-ups anticipados:

¿Qué pasa si Vault se cae? — Los clientes cachean los secretos en memoria (nunca en disco) por el TTL del lease. Si Vault está down por un período corto, los servicios siguen operando con los secretos cacheados. Vault mismo se deploya en HA con múltiples nodos y un backend de almacenamiento distribuido (Raft integrado o Consul). La probabilidad de downtime total es muy baja.

¿Cómo detectas secretos filtrados? — Herramientas como GitLeaks o TruffleHog en el pipeline de CI para escanear commits. AWS CloudTrail para detectar uso de credenciales desde IPs inesperadas. Alertas cuando una credencial se usa desde múltiples service accounts

simultáneamente.

9. (Duplicada de la 6 — la respondo ampliando con patrones adicionales)

Complementando la pregunta 6, profundizo en patrones adicionales.

Request signing para APIs internas: Cuando no es viable usar mTLS (por ejemplo, en comunicación cross-cloud o con servicios legacy), cada request interna se firma con HMAC-SHA256 usando un secreto compartido rotado desde Vault. El receptor verifica la firma antes de procesar el request. Esto previene tampering y autenticación falsificada.

Token exchange para cross-boundary calls: Cuando un servicio en el dominio A necesita llamar a un servicio en el dominio B que tiene diferente trust boundary, uso token exchange (OAuth2 Token Exchange, RFC 8693). El servicio A intercambia su token local por un token válido en el dominio B, con los scopes apropiados. Esto evita que tokens internos del dominio A sean válidos en el dominio B.

Encryption en tránsito + en payload: Más allá de TLS para el transporte, para datos sensibles (PII, datos financieros) aplico encryption a nivel de campo en el payload. Esto significa que incluso si alguien logra interceptar el tráfico TLS (por ejemplo, un proxy mal configurado que termina TLS), los campos sensibles siguen encriptados.

Idempotency keys para requests mutativas: Si un servicio A envía un POST al servicio B y la red se interrumpe después de que B procesó pero antes de que A recibió la respuesta, A reintentará. Sin idempotency keys, B procesa la operación dos veces. Cada request mutativa lleva un header Idempotency-Key: <uuid> que B usa para deduplicar.

10. How do you document architecture?

La documentación de arquitectura útil es aquella que se mantiene viva y que la gente realmente consulta. Los documentos estáticos extensos que nadie lee son un antipatrón.

Architecture Decision Records (ADRs): Cada decisión arquitectónica significativa se documenta en un ADR con estructura fija: contexto (¿qué problema estamos resolviendo?), opciones consideradas (con pros y cons de cada una), decisión (qué elegimos y por qué), y consecuencias (qué tradeoffs aceptamos). Los ADRs viven en el repositorio, versionados con git. No documentan el "qué" sino el "por qué" — el código muestra qué se hizo, el ADR explica por qué se eligió ese approach sobre los demás.

Modelo C4 para diagramas: El modelo C4 de Simon Brown es mi framework preferido para diagramación. Tiene cuatro niveles de abstracción que se alinean con diferentes audiencias. El nivel 1 (System Context) muestra el sistema como una caja negra y sus interacciones con usuarios y sistemas externos — esto es para stakeholders y PMs. El nivel 2 (Container) muestra los componentes principales (servicios, bases de datos, message brokers) y cómo se comunican — esto es para el equipo técnico. El nivel 3 (Component) muestra la estructura interna de un container (los módulos, clases principales) — esto es para los desarrolladores del servicio. El nivel 4 (Code) son

diagramas de clases, que rara vez documento formalmente porque el código es la fuente de verdad.

Diagramas como código: Uso herramientas como Structurizr (DSL para C4), Mermaid (embeddable en markdown), o PlantUML. Los diagramas se generan desde archivos de texto versionados en el repo. Esto elimina el problema de "el diagrama en Confluence no refleja la realidad" porque el diagrama se actualiza con el mismo PR que cambia el código.

API contracts como documentación viva: Los contratos de API (OpenAPI/Swagger para REST, protobuf definitions para gRPC) son la documentación más valiosa y la que más se mantiene actualizada porque el código no compila o los tests fallan si el contrato no coincide. Contract testing con Pact asegura que la documentación y la implementación siempre están sincronizadas.

Follow-ups anticipados:

¿Cómo mantienes la documentación actualizada? — Automation. Los diagramas C4 se generan desde la definición de infraestructura (Terraform, Kubernetes manifests). Los ADRs son parte del Definition of Done en los tickets. Los contratos API generan documentación automáticamente. Lo que no se puede automatizar, se review en architecture reviews trimestrales donde verificamos que los diagramas de nivel 1 y 2 sigan siendo correctos.

¿Dónde vive la documentación? — Los ADRs y diagramas-como-código viven en el monorepo. Una vista navegable se publica automáticamente (por ejemplo, con MkDocs o Backstage) para que sea accesible sin clonar el repo.

11. How do you decide between OLTP and OLAP designs when modeling data?

OLTP (Online Transaction Processing) y OLAP (Online Analytical Processing) son optimizaciones para patrones de acceso fundamentalmente diferentes, y mezclarlos en el mismo modelo es una receta para rendimiento deficiente en ambos casos.

Cuándo OLTP: Cuando el patrón dominante es leer y escribir registros individuales o pequeños conjuntos de registros, con alta concurrencia. Un sistema de pedidos donde necesitas insertar un pedido, actualizar su estado, y leer los pedidos de un usuario específico. Los modelos OLTP están normalizados (3NF o superior) para minimizar la redundancia y garantizar consistencia en escrituras. Los índices están optimizados para lookups puntuales (B-tree en columnas de filtrado como `user_id`, `order_id`). Las bases de datos típicas son PostgreSQL, MySQL, Aurora.

Cuándo OLAP: Cuando el patrón dominante es leer grandes volúmenes de datos con agregaciones (SUM, COUNT, AVG, GROUP BY) sobre columnas específicas, con poca o ninguna escritura concurrente. Un dashboard que muestra "total de ventas por región por mes en el último año". Los modelos OLAP están denormalizados — esquemas star o snowflake con fact tables y dimension tables — porque los JOINS son caros a escala y la redundancia es aceptable cuando no hay escrituras concurrentes. Los datos se almacenan en formato columnar (Parquet, ORC) y se consultan con engines como BigQuery, Redshift, Snowflake, ClickHouse.

El error común: Ejecutar queries analíticos contra la base de datos OLTP de producción. Esto degrada el rendimiento de las transacciones y puede causar locks prolongados. La solución correcta es separar los flujos: la base de datos OLTP es la fuente de verdad para operaciones, y los datos

fluyen (via CDC con Debezium, o ETL batch con dbt) hacia un data warehouse optimizado para analytics.

HTAP (Hybrid): Algunas bases de datos modernas como TiDB, CockroachDB o AlloyDB intentan servir ambos patrones. En mi experiencia, funcionan bien para cargas moderadas, pero para volúmenes realmente grandes o requerimientos de latencia estrictos, la separación sigue siendo la mejor estrategia.

Follow-ups anticipados:

¿Cómo decides el esquema del data warehouse? — Dimensional modeling (Kimball). Identifico los procesos de negocio (ventas, registros, pagos), defino las fact tables (una fila por evento medible) y las dimension tables (quién, qué, cuándo, dónde). Las métricas van en las facts, los atributos descriptivos en las dimensions.

¿Cuándo usar una base de datos columnar? — Cuando los queries típicos acceden a pocas columnas sobre muchas filas (`SELECT AVG(price) FROM sales WHERE year = 2024`). Las bases columnares leen solo las columnas necesarias, lo que reduce I/O drásticamente. Para queries que acceden a todas las columnas de pocos registros (`SELECT * FROM orders WHERE id = 123`), un formato row-based es superior.

12. How do you design a system to survive database failover without data loss?

Zero data loss durante un failover requiere que toda transacción committed en el nodo primario esté replicada en al menos un nodo secundario antes de confirmar el commit al cliente.

Replicación síncrona: En PostgreSQL, `synchronous_commit = on` con `synchronous_standby_names` configurado garantiza que cada COMMIT espere a que el WAL (Write-Ahead Log) sea escrito en al menos un standby antes de confirmar al cliente. Esto da RPO = 0 (Recovery Point Objective de cero, es decir, cero data loss), pero aumenta la latencia de escritura porque cada commit requiere un round-trip al standby. Es el tradeoff fundamental: data safety vs write performance.

Semi-síncrono como compromiso: La replicación síncrona con "remote_write" (en lugar de "remote_apply") en PostgreSQL envía el WAL al standby y espera confirmación de escritura en disco, pero no espera a que se aplique. Esto es ligeramente más rápido y aún proporciona RPO = 0, pero el standby podría estar unos segundos detrás en terms de lecturas consistentes.

Failover automático: Necesitas un mecanismo que detecte la caída del primario, promueva un standby, y redireccione el tráfico. Las opciones son Patroni (para PostgreSQL, basado en consensus con etcd/ZooKeeper), PgBouncer + Patroni (el pooler se reconfigura automáticamente), o servicios gestionados como RDS Multi-AZ que manejan esto internamente. El tiempo de failover típico es 15-30 segundos con Patroni y ~30-60 segundos con RDS Multi-AZ.

Fencing del nodo antiguo (split-brain prevention): Cuando el primario se cae y un standby es promovido, necesitas garantizar que el primario antiguo no pueda seguir aceptando escrituras si se recupera (split-brain). Patroni resuelve esto con el "leader lock" en etcd — el nodo antiguo no puede adquirir el lock y se rehúsa a operar como primario.

Punto de conexión estable: La aplicación no debe apuntar a IPs de nodos específicos. Usa un endpoint DNS (como el cluster endpoint de RDS) o un VIP (Virtual IP) gestionado por Patroni que siempre apunta al primario actual. En Kubernetes, un Service con selector que apunta al pod primario actual.

Follow-ups anticipados:

¿Cómo manejas las conexiones activas durante el failover? — Las conexiones al primario antiguo se rompen. El connection pool (HikariCP en Java) detecta las conexiones rotas, las descarta, y crea nuevas al nuevo primario. Las transacciones en vuelo fallan y deben ser reintentadas por la aplicación. Es esencial que la aplicación tenga retry logic con backoff para transacciones idempotentes.

¿Replicación síncrona no crea un punto de fallo? — Si el standby síncrono se cae, los commits en el primario se bloquean indefinidamente esperando acknowledgment. Para evitar esto, configuro múltiples standbys y `synchronous_standby_names = 'ANY 1 (standby1, standby2)'` — el commit espera a cualquiera de los dos.

13. What strategies do you use to reduce write contention at scale?

La contención de escritura ocurre cuando múltiples transacciones intentan modificar el mismo recurso simultáneamente, y solo una puede proceder mientras las demás esperan.

Sharding para distribución horizontal: Dividir los datos en shards basados en una partition key (por ejemplo, `user_id % N`) distribuye las escrituras entre múltiples nodos. Si cada nodo maneja un subconjunto de usuarios, la contención se reduce linealmente con el número de shards. La clave es elegir una partition key que distribuya uniformemente y que se alinee con los patrones de acceso — si siempre necesitas datos de un solo usuario, `user_id` es ideal; si necesitas datos cross-user frecuentemente, el sharding introduce complejidad de scatter-gather.

Append-only writes en lugar de updates in-place: En lugar de `UPDATE accounts SET balance = balance - 100 WHERE id = 42` (que lockea la fila), inserto un evento `INSERT INTO ledger (account_id, amount, type) VALUES (42, -100, 'debit')`. El balance se calcula como `SUM(amount) WHERE account_id = 42`. Los inserts no compiten entre sí porque cada fila es nueva. Periódicamente, "consolido" los eventos en un snapshot para mantener las lecturas rápidas.

Batching y buffering: Para contadores de alta frecuencia (views de un video, likes en un post), no actualizo la fila en cada evento. En su lugar, acumulo en un buffer en memoria (o Redis) y hago flush periódico (cada 5 segundos o cada 1,000 eventos) con un solo `UPDATE counter SET value = value + 1000`. Esto reduce la contención por un factor de 1000x.

CQRS (Command Query Responsibility Segregation): Separo el modelo de escritura del modelo de lectura. Las escrituras van a una store optimizada para consistencia (la base de datos transaccional), y los eventos de cambio se propagan a stores optimizadas para lectura (Elasticsearch para search, Redis para caches, materialized views para dashboards). Esto elimina la contención entre lecturas y escrituras.

Partitioned sequences: Si múltiples nodos necesitan generar IDs, un SERIAL centralizado se convierte en un bottleneck. Uso UUIDs (ULIDs si necesito order temporal), o secuencias pre-allocadas (cada nodo recibe un bloque de 10,000 IDs y genera localmente).

Follow-ups anticipados:

¿Cómo garantizas que el balance nunca sea negativo con append-only? — Uso un "reservation" pattern: antes de debitar, inserto un registro de reservación que reduce el balance disponible. Si la reservación más el balance actual sería negativo, la rechazo. Los registros de reservación tienen TTL — si no se confirman, expiran y el balance se libera.

14. When do you choose to denormalize intentionally — and how do you keep it consistent?

La denormalización es una decisión consciente de duplicar datos para optimizar lecturas, aceptando la complejidad adicional de mantener la consistencia.

Cuándo denormalizar: Cuando un query de lectura frecuente (paths hot del sistema) requiere JOINS costosos que degradan la latencia, y la relación lectura/escritura es alta (100:1 o más). El ejemplo clásico: un servicio de ecommerce muestra el nombre del vendedor junto a cada producto. Normalizado, esto requiere un JOIN entre `products` y `sellers` en cada page view.

Denormalizado, `seller_name` se almacena directamente en la tabla `products`, y las lecturas son un simple lookup sin JOIN.

Cuándo NO denormalizar: Cuando la relación escritura/lectura es alta, cuando los datos denormalizados cambian frecuentemente, o cuando la consistencia eventual no es aceptable. Si el nombre del vendedor cambia 10 veces al día y necesitas que se refleje inmediatamente en todos los productos, la denormalización crea más problemas de los que resuelve.

Patrones para mantener consistencia:

La primera opción es actualización dual en la misma transacción: cuando actualizas el dato canónico, actualizas también todas las copias denormalizadas en la misma transacción. Funciona solo si todo está en la misma base de datos. Es simple pero crea acoplamiento.

La segunda opción es CDC + event processing: usas Change Data Capture (Debezium) para capturar cambios en la tabla canónica y un consumidor actualiza las copias. Es eventualmente consistente (latencia de segundos), pero desacoplado y resiliente.

La tercera opción es transactional outbox: cuando actualizas el dato canónico, en la misma transacción insertas un evento en una tabla outbox. Un poller o CDC lee la outbox y propaga el cambio a las copias. Esto garantiza at-least-once delivery sin depender de distributed transactions.

Follow-ups anticipados:

¿Cómo detectas inconsistencias? — Reconciliation jobs periódicos. Un job nocturno compara los datos denormalizados contra la fuente canónica y genera alertas o correcciones automáticas para discrepancias. Esto es un safety net, no el mecanismo principal de sincronización.

¿Cuánta latencia es aceptable? — Depende del dominio. Para un nombre de usuario en posts de un

feed, segundos de delay son imperceptibles. Para un precio de producto en un carrito de compras, el dato debe estar fresco al momento del checkout — en ese caso, re-verifico contra la fuente canónica en el momento crítico.

15. How do you architect a system to handle traffic spikes 100x its normal load?

Un spike de 100x es un evento extremo que no se maneja simplemente agregando más servidores — requiere decisiones arquitectónicas previas que hagan el sistema elástico por diseño.

Auto-scaling con pre-warming: El auto-scaling reactivo (agregar instancias cuando la CPU sube) es insuficiente para spikes de 100x porque el provisioning tarda minutos y el spike puede durar segundos. Para eventos predecibles (Black Friday, lanzamiento de producto), hago pre-scaling: aumento la capacidad anticipadamente basándome en proyecciones. Para spikes impredecibles, necesito un sistema que pueda absorber la carga con los recursos existentes.

Queueing como amortiguador: El patrón fundamental es poner un message queue (SQS, Kafka) entre el ingreso de requests y el procesamiento. El frontend acepta el request inmediatamente (responde 202 Accepted), encola el trabajo, y el backend procesa a su propio ritmo. Si llegan 100x requests, la cola crece pero no se cae nada — solo aumenta la latencia de procesamiento. Esto requiere que la operación pueda ser asíncrona (no funciona para reads que necesitan respuesta inmediata).

Caching agresivo: Para reads, un CDN (CloudFront, Fastly) absorbe la mayoría del tráfico si el contenido es cacheable. Para APIs, un cache de application level (Redis) con TTL corto (30-60 segundos) puede reducir la carga a la base de datos en 95%+. El patrón cache-aside con stale-while-revalidate es ideal: sirve el cache stale inmediatamente mientras refresca en background.

Load shedding inteligente: Cuando el sistema está saturado, es mejor rechazar algunas requests limpiamente (429 Too Many Requests con Retry-After header) que degradarse de manera impredecible para todos. Implemento un mecanismo de admission control que mide la latencia p99 interna, y cuando supera un threshold, comienza a rechazar un porcentaje de requests no-críticas (queries de analytics, features opcionales) para proteger el core (checkout, pagos).

Degradación graceful: Diseño el sistema con "feature flags de degradación" que puedo activar bajo carga extrema. Ejemplo: desactivar recomendaciones personalizadas y mostrar las top-10 genéricas (pre-cacheadas), desactivar búsqueda full-text y caer a búsqueda por categoría, mostrar precios cacheados en lugar de calcular descuentos dinámicos.

Follow-ups anticipados:

¿Cómo testas la resiliencia a 100x? — Load testing con herramientas como k6 o Locust simulando el patrón de tráfico esperado. Chaos engineering con LitmusChaos o AWS FIS para inyectar fallos durante el load test. El objetivo no es que nada falle, sino que falle de forma controlada.

¿Qué base de datos aguanta 100x? — Ninguna base de datos relacional convencional escala 100x instantáneamente. Por eso el caching y el queueing son esenciales. DynamoDB on-demand es una de las pocas opciones que escala automáticamente sin pre-provisioning, pero el costo puede ser significativo.

16 y 24. What signals tell you the application tier, storage tier, or network is the bottleneck?

(Estas preguntas son idénticas en la imagen, así que respondo una vez con profundidad.)

Diagnosticar el bottleneck requiere observar las señales correctas en cada capa y descartarlas sistemáticamente.

Señales de bottleneck en la application tier: La CPU de los pods/instancias está alta (>80% sostenido) pero la base de datos y la red están holgadas. La latencia p99 es alta pero los query times en la DB son bajos — el tiempo se consume en la aplicación (serialización, procesamiento, GC pauses). En Java, los GC logs muestran Full GC frecuentes o pauses largos. Los thread dumps muestran threads bloqueados esperando CPU o locks internos de la aplicación. El heap de la JVM está constantemente cerca del máximo. El throughput del servicio no mejora al agregar más réplicas (lo que indica un bottleneck per-instance, no de capacidad total).

Señales de bottleneck en el storage tier: Los query times en la base de datos son altos. En PostgreSQL, `pg_stat_statements` muestra queries con `mean_time` alto. `pg_stat_activity` muestra muchas conexiones en estado `active` o `idle in transaction`. Los IOPS del disco están en su límite (visible en CloudWatch para RDS o en `iostat` para self-hosted). El CPU del servidor de base de datos está alto por queries complejos o full table scans. El `wait_event_type` en `pg_stat_activity` muestra `IO` o `Lock` frecuentemente. El connection pool de la aplicación (HikariCP) muestra alto `pending` — las conexiones están saturadas.

Señales de bottleneck en la red: La latencia entre servicios (visible en traces distribuidos) es alta, pero el processing time de cada servicio individual es bajo. Packet loss o retransmissions (visible con `netstat -s` o las métricas del load balancer). DNS resolution lenta. TCP connection establishment lenta (alto `time-to-first-byte` sin alto processing time). El throughput del Network Interface está cerca de su límite (en AWS, las instancias tienen límites de bandwidth por tipo).

Metodología de diagnóstico: Sigo el método USE (Utilization, Saturation, Errors) de Brendan Gregg. Para cada recurso (CPU, memoria, red, disco), mido qué tan utilizado está, qué tan saturado (queue depth), y cuántos errores genera. El recurso con la saturación más alta es el bottleneck.

Follow-ups anticipados:

¿Qué herramientas usas? — Grafana dashboards con métricas de Prometheus para la visión general. Para deep dive: `pg_stat_statements` y `EXPLAIN ANALYZE` para DB, `async-profiler` o JFR (Java Flight Recorder) para la aplicación, `tcpdump` y `mtr` para red.

17. How do you design components to avoid single points of failure?

Un SPOF (Single Point of Failure) es cualquier componente cuya caída detiene el sistema completo.

Eliminar SPOFs es una disciplina de diseño, no algo que se agrega después.

Redundancia en cada capa: Cada componente crítico debe tener al menos dos instancias en ejecución simultánea, idealmente en zonas de disponibilidad (AZ) diferentes. La aplicación corre con mínimo 2 réplicas detrás de un load balancer. La base de datos tiene al menos un standby en otra AZ con failover automático (RDS Multi-AZ, Patroni). El message broker (SQS, Kafka) está replicado nativamente. El load balancer mismo es un servicio distribuido (ALB de AWS opera en múltiples AZs automáticamente).

Stateless services: Los servicios de aplicación no deben almacenar estado en memoria o disco local. Todo el estado vive en stores externos (base de datos, Redis, S3). Esto permite que cualquier instancia sea reemplazada sin pérdida de datos. Las sesiones de usuario, si existen, van en Redis, no en memoria del servidor.

Circuit breakers para aislamiento de fallos: Si el servicio de recomendaciones se cae, el checkout no debe caerse. Implemento circuit breakers (Resilience4j en Java) que detectan cuando un servicio downstream está fallando y "abren el circuito", retornando un fallback inmediatamente en lugar de esperar y propagar el timeout. Los estados son: cerrado (operación normal), abierto (fallback), y semi-abierto (probando si el servicio se recuperó).

Bulkheads para aislamiento de recursos: Asigno thread pools o connection pools separados para diferentes funcionalidades críticas. Si el pool de conexiones para "búsqueda de productos" se agota por un spike, el pool para "procesamiento de pagos" no se ve afectado. Esto previene que un fallo en una funcionalidad no-crítica derribe una crítica.

DNS como SPOF oculto: DNS es frecuentemente un SPOF que se ignora. Uso múltiples proveedores de DNS (Route53 + un secundario) con health checks y failover. Los TTL de los registros DNS son suficientemente bajos (60-300s) para que los cambios se propaguen rápidamente, pero no tan bajos que generen una carga excesiva de queries DNS.

Follow-ups anticipados:

¿Y el orquestador (Kubernetes)? — El control plane de Kubernetes es un SPOF si se auto-gestiona. Por eso uso EKS/GKE donde el control plane es gestionado y distribuido por el cloud provider. Los worker nodes están en múltiples AZs con auto-scaling groups. Si una AZ se cae, Kubernetes re-programa los pods en las AZs restantes.

18. What patterns help eliminate shared state between nodes?

El estado compartido entre nodos introduce acoplamiento, contención, y puntos de fallo. Eliminarlo es clave para escalabilidad horizontal.

Event sourcing: En lugar de almacenar el estado actual de una entidad, almaceno la secuencia de eventos que lo produjeron. Cada nodo puede reconstruir el estado reproduciendo los eventos desde un event store. No hay "estado compartido" porque cada nodo calcula su propia vista del estado. El event store es append-only, eliminando contención de escritura. El tradeoff es complejidad: reconstruir el estado requiere replay, lo que se mitiga con snapshots periódicos.

CRDTs (Conflict-free Replicated Data Types): Para datos que necesitan ser modificados concurrentemente por múltiples nodos sin coordinación, los CRDTs garantizan convergencia

eventual sin conflictos. Un G-Counter (grow-only counter) distribuido funciona así: cada nodo mantiene su propio contador, y el total es la suma de todos. Un nodo puede incrementar su contador sin consultar a nadie. Periódicamente, los nodos intercambian sus valores y toman el máximo de cada componente. Esto converge automáticamente sin conflictos. Redis usa CRDTs internamente para replicación multi-master en Redis Enterprise.

Sticky sessions como antipatrón: Las sticky sessions (el load balancer envía siempre al mismo nodo al mismo usuario) parecen eliminar el problema de estado compartido, pero en realidad crean un SPOF por usuario y previenen el auto-scaling efectivo. Son un parche, no una solución. La solución real es externalizar el estado de sesión a Redis.

Partitioned processing: En lugar de que N nodos compitan por procesar los mismos datos, particiono el trabajo. En Kafka, cada partición es procesada por un solo consumidor en el grupo. No hay estado compartido porque cada consumidor "posee" sus particiones. Si un consumidor se cae, Kafka reasigna sus particiones a otro consumidor, que reconstruye su estado desde el offset más reciente.

Follow-ups anticipados:

¿Cuándo es inevitable el estado compartido? — Cuando necesitas coordinación (elección de líder, distributed locks, secuencias ordenadas). Para esos casos, uso un servicio de coordinación como ZooKeeper o etcd que está diseñado específicamente para manejar estado compartido con garantías de consistencia. La clave es minimizar qué estado pasa por ese servicio — solo metadata de coordinación, nunca datos de negocio.

19. What's your approach to rate limiting at API, service, and user levels?

El rate limiting protege el sistema de abuso, distribuye recursos equitativamente, y previene cascading failures. Se aplica en múltiples capas con diferentes granularidades.

En el API Gateway (capa más externa): Es la primera línea de defensa. Aplico rate limiting global (requests por segundo totales que el sistema puede manejar) y por API key o IP. El algoritmo que prefiero es Token Bucket: cada API key tiene un bucket con N tokens (la capacidad máxima, que permite bursts) que se reabastece a una tasa de R tokens por segundo (la tasa sostenida). Cada request consume un token. Si el bucket está vacío, el request se rechaza con 429. Esto permite bursts cortos (100 requests instantáneos si el bucket estaba lleno) pero limita la tasa sostenida.

Por servicio (capa intermedia): Cada microservicio tiene su propio rate limit para protegerse de otros servicios internos que podrían sobrecargarlo. Un servicio de notificaciones podría limitar a 1,000 requests/segundo totales, independientemente de quién los envíe. Esto previene que un bug en el servicio de pedidos (un loop infinito de notificaciones) derribe el servicio de notificaciones.

Por usuario (capa más granular): Limito las operaciones costosas por usuario para prevenir abuso individual. Búsquedas: 30/minuto. Creación de recursos: 10/minuto. Login attempts: 5/5-minutos. Estos se implementan con Redis usando keys como `rate:user:{id}:search` con TTL. El algoritmo Sliding Window Log es más preciso que Fixed Window para esto: almaceno el timestamp de cada request en un sorted set de Redis, y cuento cuántos hay en la ventana actual.

Implementación distribuida: En un sistema con múltiples instancias, el rate limiting debe ser centralizado (Redis) para ser preciso. Un rate limiter local por instancia subestima el rate total porque cada instancia solo ve su fracción de tráfico. Uso Redis con MULTI/EXEC o Lua scripts para operaciones atómicas de check-and-increment.

Response headers para transparencia: Siempre incluyo headers informativos: X-RateLimit-Limit (el límite), X-RateLimit-Remaining (requests restantes), X-RateLimit-Reset (cuándo se resetea, en Unix timestamp), y Retry-After en la respuesta 429. Esto permite que los clientes implementen backoff inteligente.

Follow-ups anticipados:

¿Cómo manejas rate limiting para clientes premium vs free? — Tiers de rate limiting. Cada plan tiene sus propios límites almacenados en un servicio de configuración. El gateway consulta el plan del API key y aplica los límites correspondientes. Los cambios de plan se reflejan inmediatamente sin redeploy.

¿Qué pasa si Redis se cae? — Fail-open (permitir tráfico) vs fail-closed (rechazar tráfico). Para la mayoría de los casos, fail-open es preferible: es mejor permitir algo de tráfico extra por unos minutos que rechazar todo. Pero para endpoints sensibles (login, pagos), puede ser necesario fail-closed con un rate limiter local como fallback.

20. What patterns help make horizontal scaling predictable and cost-efficient?

La escalabilidad horizontal (agregar más nodos) es predecible solo si el sistema está diseñado para que cada nodo adicional aporte capacidad proporcional.

Servicios stateless: La condición fundamental. Si un servicio almacena estado en memoria, no puedes agregar instancias arbitrariamente porque el estado no se comparte. Externaliza todo el estado a stores apropiados (Redis para sesiones, PostgreSQL para datos persistentes, S3 para archivos). Un servicio stateless puede escalarse simplemente aumentando el `replicas` en Kubernetes.

Escalamiento basado en métricas de negocio, no de infraestructura: Escalar por CPU o memoria es reactivo y a menudo tardío. Prefiero escalar por métricas de negocio: requests en cola (queue depth de SQS), latencia p99 del endpoint, o tasa de requests por segundo. KEDA (Kubernetes Event-Driven Autoscaler) permite escalar pods basándose en métricas externas como la profundidad de una cola SQS.

Right-sizing de instancias: Usar instancias grandes ("vertical") es más simple pero ineficiente — pagas por capacidad que no usas fuera de peak. Usar muchas instancias pequeñas ("horizontal") es más eficiente pero introduce overhead de coordinación. El sweet spot depende del servicio: servicios CPU-bound se benefician de instancias con muchos cores; servicios memory-bound necesitan instancias con más RAM. En AWS, Graviton instances (ARM) ofrecen ~20% mejor precio/rendimiento que x86 equivalentes.

Predictive scaling: AWS ofrece predictive auto-scaling que analiza patrones históricos de tráfico y pre-escala antes de que el spike ocurra. Para patrones diarios predecibles (más tráfico de 9am a

6pm), esto es más eficiente que reactive scaling.

Spot/Preemptible instances para workloads tolerantes: Para procesamiento batch, workers de colas, o workloads que pueden tolerar interrupciones, uso spot instances que cuestan 60-90% menos. El servicio debe manejar graceful shutdown cuando la instancia es reclamada (draining de connections, checkpoint del progreso).

Cost allocation y chargeback: Etiqueto todos los recursos con `team`, `service`, y `environment`. Esto permite saber exactamente cuánto cuesta cada servicio y qué equipo está generando el gasto, habilitando decisiones informadas de optimización.

Follow-ups anticipados:

¿Cómo evitas escalar innecesariamente? — Cooldown periods entre scaling events (no escalar arriba y abajo en minutos). Mínimo de instancias que cubra el baseline traffic. Scaling policies con thresholds diferentes para scale-up (agresivo, 60% CPU) y scale-down (conservador, 30% CPU durante 15 minutos).

21. How do you design workloads to run across multiple availability zones?

Multi-AZ es el estándar mínimo para alta disponibilidad. Una AZ es un data center (o grupo de data centers) en una región de cloud que opera independientemente — tiene su propia red, energía, y refrigeración. Si una AZ se cae, las otras siguen operando.

Distribución uniforme de compute: Los pods de Kubernetes o las instancias EC2 deben distribuirse uniformemente entre al menos 2, idealmente 3, AZs. En Kubernetes, uso `topologySpreadConstraints` para garantizar que los pods se distribuyan equitativamente:

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
```

En EC2, el Auto Scaling Group se configura con `subnets` en múltiples AZs y `capacity-rebalancing` habilitado.

Bases de datos Multi-AZ: RDS Multi-AZ mantiene un standby síncrono en otra AZ con failover automático. Aurora distribuye el storage entre 3 AZs con 6 copias de los datos (2 por AZ) y puede perder una AZ completa sin pérdida de datos. Para bases de datos auto-gestionadas, el standby síncrono de Patroni se configura en una AZ diferente al primario.

Tráfico cross-AZ y costo: El tráfico entre AZs en AWS cuesta dinero (~\$0.01/GB en cada dirección). Para servicios con alto volumen de comunicación inter-servicio, esto puede ser significativo. Optimización: usar zone-aware routing en el service mesh (Istio locality-aware routing) para que un servicio en la AZ-a prefiera llamar a instancias en la AZ-a, y solo cruce AZ si no hay instancias saludables localmente.

Almacenamiento: S3 es multi-AZ por defecto (los datos se replican en al menos 3 AZs). EBS volumes son single-AZ, así que si necesitas storage persistente multi-AZ, usa EFS o diseña la

aplicación para reconstruir el estado desde una fuente durable.

Follow-ups anticipados:

¿Multi-AZ o multi-region? — Multi-AZ protege contra fallas de un data center individual. Multi-region protege contra fallas de toda una región (evento extremadamente raro pero catastrófico), y también reduce latencia para usuarios geográficamente distribuidos. Multi-region es significativamente más complejo porque la latencia entre regiones es de 50-200ms (vs 1-2ms entre AZs), lo que hace impracticable la replicación síncrona de base de datos. Solo voy a multi-region si hay un requerimiento de negocio explícito (compliance regulatorio, latencia global, o RPO/RTO extremo).

22. What are the trade-offs between managed services and self-hosted components in the cloud?

Esta es una decisión que afecta no solo el costo monetario, sino el costo operacional, la velocidad de desarrollo, y la flexibilidad arquitectónica.

Managed services (RDS, ElastiCache, SQS, etc.): La ventaja principal es que el cloud provider maneja patching, backups, failover, scaling, y monitoring. Esto libera al equipo de operaciones para enfocarse en diferenciadores de negocio. Para un equipo pequeño (<10 ingenieros), self-hosting una base de datos PostgreSQL es irresponsable — no tienes la capacidad operacional para hacer on-call de base de datos 24/7. El costo es que pierdes control: no puedes tunear parámetros específicos del kernel, estás limitado a las versiones y extensiones que el provider soporta, y el pricing es típicamente 2-5x más caro que el cómputo raw.

Self-hosted: Cuando necesitas control total sobre la configuración, extensiones custom (PostgreSQL con PostGIS, TimescaleDB, pg_partman que no esté soportada en RDS), o cuando el volumen es tan alto que el pricing de managed services se vuelve prohibitivo. También cuando tienes regulaciones que requieren hardware dedicado o ubicaciones específicas que el managed service no ofrece.

Mi framework de decisión: Empiezo siempre con managed services. El overhead de self-hosting es significativo y a menudo subestimado — no es solo el setup inicial, sino el mantenimiento continuo: security patches, major version upgrades, monitoring, alerting, capacity planning, backup testing, disaster recovery drills. Solo paso a self-hosted cuando hay una razón técnica o de costo que lo justifique claramente, y cuando el equipo tiene la madurez operacional para manejarlo.

El antipatrón "Not Invented Here": He visto equipos que self-hostean todo (Kafka, Elasticsearch, Redis, PostgreSQL) "para tener control" y terminan gastando más de la mitad del tiempo de ingeniería en operaciones en lugar de en producto. Hay una diferencia entre "podemos hacerlo" y "deberíamos hacerlo".

Follow-ups anticipados:

¿Cuándo es más barato self-hosted? — Típicamente cuando el volumen es predecible y alto. Un RDS r6g.2xlarge con Multi-AZ cuesta ~\$1,500/mes. Dos EC2 r6g.2xlarge con Patroni + EBS cuestan ~\$800/mes, pero necesitas contar el costo del tiempo de ingeniería para configurar, mantener, y hacer on-call. Si el ingeniero cuesta \$100/hora y gasta 10 horas/mes en la DB, el costo

real es \$1,800, más caro que RDS.

¿Qué servicios siempre uso managed? — Bases de datos (RDS/Aurora), message queues (SQS), object storage (S3), DNS (Route53), load balancers (ALB). Estos son commodities donde el valor diferencial de self-hosting es mínimo.

23. How do you handle atomic updates when NoSQL systems don't support multi-document transactions?

En bases de datos NoSQL como DynamoDB o la mayoría de configuraciones de MongoDB (pre-4.0), las transacciones multi-documento no existen o son limitadas. Necesitas patrones alternativos para mantener consistencia.

Aggregate design (el más importante): El patrón fundamental de DDD dice que un aggregate es una unidad de consistencia transaccional. Si necesitas actualizar dos entidades atómicamente, pertenecen al mismo aggregate y deben estar en el mismo documento/item. En DynamoDB, un pedido con sus líneas de detalle se modela como un solo item (o se usa una transacción de DynamoDB `TransactWriteItems` con hasta 100 items). Diseñar los aggregates correctamente elimina la mayoría de las necesidades de transacciones multi-documento.

Saga pattern para transacciones distribuidas: Cuando necesitas coordinar cambios entre múltiples aggregates o servicios, usas una Saga: una secuencia de transacciones locales donde cada paso tiene una operación compensatoria. Si el paso 3 falla, ejecutas las compensaciones de los pasos 2 y 1 en orden inverso. Hay dos variantes: coreografía (cada servicio escucha eventos y reacciona) y orquestación (un servicio coordinador dirige la secuencia). La orquestación es más fácil de debuggear y monitorear.

Idempotent operations con version checks: Cada documento tiene un campo `version`. Para actualizar, lees la versión actual, haces el cambio, y escribes condicionalmente (`IF version = N THEN update AND set version = N+1`). En DynamoDB, esto es `ConditionExpression: 'version = :currentVersion'`. Si la condición falla, otro proceso modificó el documento y debes reintentar.

Change Data Capture para sincronización eventual: Si necesitas que un cambio en el Servicio A se refleje en el Servicio B, y ambos tienen su propia store, usas CDC o un event emitido por A que B consume. La consistencia es eventual, pero la experiencia de usuario se maneja con técnicas como "read-your-own-writes" (después de una escritura, el usuario lee de la fuente primaria, no de la réplica).

Follow-ups anticipados:

¿Cuándo es mejor usar SQL? — Si la mayoría de tus operaciones requieren transacciones ACID multi-tabla, una base de datos relacional es la herramienta correcta. NoSQL brilla cuando el modelo de datos se alinea naturalmente con documentos autocontenidos (catálogos de productos, perfiles de usuario, configuraciones), y la escala horizontal es un requerimiento. No uses NoSQL solo porque es "moderno"; úsalo porque el patrón de acceso lo justifica.

¿DynamoDB tiene transacciones? — Sí, desde 2018. `TransactWriteItems` soporta hasta 100 items en una transacción ACID. Pero tiene limitaciones: todos los items deben estar en la misma

región, la transacción no puede exceder 4MB, y el costo es el doble de una operación normal (se cobran 2x WCU). Para muchos casos de uso, es suficiente.

Nota sobre preguntas duplicadas

La pregunta 9 es idéntica a la 6 ("What's your process for securing internal service-to-service communication?"), y la pregunta 24 es idéntica a la 16 ("What signals tell you the application tier, storage tier, or network is the bottleneck?"). Las he respondido ampliando con contenido complementario donde aplica.

Parte 2: Modular Monolith y Spring Modulith — 20 Preguntas para Senior Java / Spring Boot

Contexto: Amazon Prime Video migró de microservicios de vuelta a un monolito y redujo costos un 90%. Estas preguntas exploran por qué el "modular monolith" es frecuentemente la decisión arquitectónica correcta, y cómo implementarlo con Spring Modulith.

1. Modulith vs Monolith vs Microservices — when to choose which

Estas tres arquitecturas no son puntos en una línea de progresión donde una es "mejor" que la otra. Son herramientas para problemas diferentes, y la elección depende de la madurez del equipo, el tamaño de la organización, y los requerimientos de escalabilidad independiente.

Monolito clásico (big ball of mud): Una sola aplicación donde todos los módulos están acoplados sin fronteras claras. Cualquier clase puede importar cualquier otra clase. El esquema de base de datos es un grafo denso de foreign keys entre todas las tablas. Esto no es inherentemente malo para un equipo pequeño (2-5 devs) trabajando en un producto en fase temprana. La velocidad de desarrollo es máxima porque no hay overhead de comunicación inter-proceso, no hay contratos de API que negociar, no hay network latency, y el debugging es trivial (un solo stack trace). El problema aparece cuando el equipo crece a 15-20+ personas y los cambios en una parte del sistema rompen otra parte de forma impredecible. Esto no es un problema del monolito como patrón de deployment — es un problema de falta de fronteras internas.

Modular monolith (modulith): Una sola aplicación deployable, pero con fronteras bien definidas entre módulos. Cada módulo encapsula su dominio, expone una API explícita, y no accede directamente a los internals de otros módulos. Los módulos se comunican a través de interfaces públicas y eventos de dominio. El esquema de base de datos puede estar particionado por módulo (cada módulo posee sus tablas, otros módulos no hacen JOINS directos contra ellas). Es la opción correcta para la mayoría de las organizaciones con 5-50 ingenieros, y es el punto de partida correcto incluso si anticipas que eventualmente necesitarás microservicios, porque las fronteras del modulith se convierten naturalmente en las fronteras de los futuros microservicios.

Microservicios: Múltiples aplicaciones deployables independientemente, cada una con su propio proceso, base de datos, y ciclo de release. La ventaja real (y frecuentemente la única que justifica la complejidad) es el deployment independiente: el equipo de Pagos puede deployar sin coordinar con el equipo de Inventario. Esto habilita autonomía organizacional. El costo es enorme: necesitas service discovery, load balancing, circuit breakers, distributed tracing, message brokers, contract testing, y un equipo de plataforma que mantenga toda esa infraestructura. El caso de Prime Video demostró que si un solo equipo posee todos los servicios, los microservicios son overhead puro sin beneficio organizacional.

Mi heurística de decisión: Si un solo equipo puede ownar todo el sistema, modolith. Si múltiples equipos necesitan deployar independientemente con diferentes cadencias de release, microservicios para los componentes que lo requieran. Nunca microservicios "por si acaso" — la complejidad distribuida es un costo real que pagas desde el día 1, mientras el beneficio de escala independiente puede no materializarse nunca.

Follow-ups anticipados:

¿El modolith no se convierte eventualmente en un big ball of mud? — Solo si no enforces las fronteras. Spring Modulith con `ApplicationModules.verify()` en el CI previene violaciones de fronteras a nivel de compilación. Un monolito clásico no tiene esa herramienta — depende de la disciplina humana, que siempre falla a largo plazo.

¿Y si un módulo necesita escalar independientemente? — Extraes ese módulo específico como microservicio cuando la data lo justifica. Un modolith bien diseñado hace esa extracción relativamente sencilla porque las fronteras ya están definidas. Esto es fundamentalmente más seguro que diseñar microservicios desde el inicio basándote en suposiciones de escala que podrían ser incorrectas.

2. What is Spring Modulith and what problem does it actually solve?

Spring Modulith es un framework de Pivotal/VMware que proporciona herramientas para implementar y enforzar modular monoliths en aplicaciones Spring Boot. No es un runtime framework — es un conjunto de convenciones, verificaciones en tiempo de compilación, y herramientas de observabilidad que hacen que las fronteras entre módulos sean explícitas y verificables.

El problema que resuelve: En un monolito Spring Boot típico, cualquier `@Component` puede inyectar cualquier otro `@Component` porque todo vive en el mismo `ApplicationContext`. No hay mecanismo built-in para decir "esta clase es interna al módulo de Pagos y no debería ser accesible desde el módulo de Pedidos". Los desarrolladores crean dependencias ad-hoc entre packages, y con el tiempo, el grafo de dependencias se convierte en un espagueti imposible de refactorizar. Spring Modulith resuelve esto proporcionando una forma de definir módulos, verificar que las reglas de acceso se cumplan, y comunicar entre módulos de forma desacoplada.

Lo que ofrece concretamente:

Primero, un modelo de módulos basado en convención: cada package de primer nivel bajo el

package base de la aplicación se considera un módulo. Si tu aplicación es `com.myapp`, entonces `com.myapp.orders`, `com.myapp.payments`, y `com.myapp.inventory` son tres módulos.

Segundo, verificación de fronteras:

`ApplicationModules.of(Application.class).verify()` analiza el grafo de dependencias en compile time y falla si un módulo accede a clases internas de otro módulo (subpackages que no están explícitamente expuestos).

Tercero, eventos de dominio con entrega garantizada: un Event Publication Registry que persiste los eventos en la base de datos antes de entregarlos, garantizando at-least-once delivery sin necesidad de Kafka u otro message broker externo.

Cuarto, testing aislado: `@ApplicationModuleTest` bootstrapea solo el módulo bajo prueba y sus dependencias declaradas, haciendo que los tests de integración sean rápidos.

Quinto, documentación auto-generada: genera diagramas C4 y Application Module Canvas a partir del código, manteniéndolos siempre actualizados.

Follow-ups anticipados:

¿Spring Modulith es production-ready? — Sí. Desde la versión 1.0 (2023) es GA. Empresas como Zalando y varias organizaciones en el ecosistema Spring lo usan en producción. La adopción incremental es posible: puedes agregar Spring Modulith a un monolito existente y empezar a enforced fronteras gradualmente.

¿Requiere cambiar la estructura del proyecto? — Si ya sigues la convención de un package por dominio, el cambio es mínimo. Si tu estructura es "por capa" (`controllers/`, `services/`, `repositories/`), necesitas reorganizar hacia "por feature/dominio", lo cual es un refactoring significativo pero beneficioso independientemente de si usas Spring Modulith o no.

3. How do you define and detect modules in Spring Modulith?

Spring Modulith usa un modelo convencional para detectar módulos, pero también ofrece customización explícita cuando las convenciones no son suficientes.

Detección por convención (el default): Cada package directo bajo el package de la clase `@SpringBootApplication` se detecta como un módulo. Si la aplicación está en `com.example.shop`, entonces `com.example.shop.orders` es el módulo "orders" y `com.example.shop.catalog` es el módulo "catalog". Todo lo que está dentro de sub-packages (como `com.example.shop.orders.internal.validation`) se considera interno al módulo y no es accesible desde otros módulos.

```
// Detectar y listar todos los módulos
ApplicationModules modules = ApplicationModules.of(ShopApplication.class);
modules.forEach(module -> System.out.println(module.getName()));
// Output: orders, catalog, payments, shipping
```

API pública vs internal: El package raíz del módulo (`com.example.shop.orders`) es la API pública. Todo en sub-packages (`com.example.shop.orders.internal`,

`com.example.shop.orders.infrastructure`) es interno. Otro módulo puede depender de `OrderService` si está en `com.example.shop.orders`, pero no de `OrderRepositoryImpl` si está en `com.example.shop.orders.internal`.

Módulos explícitos con `@ApplicationModule`: Cuando la convención no es suficiente, puedes anotar un `package-info.java`:

```
// En com/example/shop/orders/package-info.java
@org.springframework.modulith.ApplicationModule(
    allowedDependencies = {"catalog", "payments"}
)
package com.example.shop.orders;
```

Esto declara explícitamente que el módulo "orders" solo puede depender de "catalog" y "payments". Si un desarrollador agrega una dependencia a "shipping", `verify()` falla en el CI.

Módulos anidados: Si necesitas una estructura más profunda, puedes tener módulos dentro de módulos usando `@ApplicationModule(type = Type.OPEN)` para módulos que exponen todo su contenido, o la convención de named interfaces para control fino.

Follow-ups anticipados:

¿Cómo manejas código compartido (utilities, commons)? — Un módulo `shared` o `commons` con DTOs, value objects, y utilidades que otros módulos necesitan. Pero con cuidado: si `commons` crece demasiado, se convierte en un "god module" que acopla todo. Prefiero que el shared code sea minimal (value objects de dominio compartido, excepciones base) y que cada módulo tenga sus propias utilidades internas.

¿Puedo tener múltiples módulos en el mismo package? — No con la convención default. Cada package de primer nivel es un módulo. Si necesitas subdividir, usa sub-packages dentro del módulo o reorganiza la estructura de packages.

4. Module encapsulation: how do you hide internals from other modules?

La encapsulación de módulos en Spring Modulith se basa en la visibilidad de packages, reforzada con verificación estática. El objetivo es que solo las clases explícitamente diseñadas como API sean accesibles desde fuera del módulo.

El mecanismo fundamental: Todo lo que está en el package raíz del módulo es público (la API). Todo lo que está en sub-packages es interno. Por ejemplo, en el módulo "orders":

```
com.example.shop.orders/
├── OrderService.java      ← API pública (otros módulos pueden usarla)
├── OrderCreatedEvent.java ← API pública (evento de dominio)
├── internal/
│   ├── OrderRepository.java ← Interno (solo accesible dentro de "orders")
│   ├── OrderValidator.java  ← Interno
│   └── OrderEntity.java     ← Interno
└── infrastructure/
    ├── OrderJpaAdapter.java ← Interno
    └── OrderKafkaPublisher.java ← Interno
```

```
verify() garantiza que ningún otro módulo importe clases de  
com.example.shop.orders.internal o  
com.example.shop.orders.infrastructure.
```

¿Por qué no basta con el access modifier de Java? Porque `public` en Java es "público para todo el classpath". No hay un mecanismo built-in para decir "público dentro de mi módulo pero privado para otros módulos". Java 9 introdujo el module system (JPMS), pero su adopción en aplicaciones Spring Boot es prácticamente nula por la complejidad de configuración y las incompatibilidades con el ecosistema de libraries. Spring Modulith logra lo mismo con convenciones de packages y verificación estática, sin modificar la JVM.

Reforzamiento en CI: La verificación se ejecuta como un test:

```
@Test  
void verifyModularStructure() {  
    ApplicationModules.of(ShopApplication.class).verify();  
}
```

Si alguien agrega un `import com.example.shop.orders.internal.OrderEntity` en el módulo de shipping, este test falla en el CI, bloqueando el merge. Esto es enforcement automático — no depende de code review humano.

Follow-ups anticipados:

¿No es esto lo mismo que ArchUnit? — ArchUnit permite definir reglas arquitectónicas custom ("las clases en el package X no deben depender del package Y"), pero requiere que escribas las reglas manualmente. Spring Modulith provee las reglas automáticamente basándose en la estructura de módulos. Puedes usar ambos: Spring Modulith para las reglas de módulos, y ArchUnit para reglas adicionales específicas del proyecto (naming conventions, layering dentro de un módulo).

¿Cómo expongo un DTO a otro módulo sin exponer la entity? — El DTO vive en el package raíz del módulo (API pública), y la entity vive en un sub-package (internal). El service del módulo convierte entity → DTO antes de retornarlo. Esto es exactamente el patrón que usarías si fueran microservicios, pero sin serialización de red.

5. Named interfaces — how do you expose APIs between modules cleanly?

Las named interfaces son el mecanismo de Spring Modulith para exponer sub-packages específicos como parte de la API pública de un módulo, yendo más allá de la convención de "solo el package raíz es público".

El problema que resuelven: A veces el package raíz del módulo se vuelve demasiado grande si todas las clases públicas deben estar ahí. Quieres organizar tu API en sub-packages temáticos (por ejemplo, `orders.api`, `orders.events`, `orders.spi`) sin que Spring Modulith los trate como internos.

Implementación: Creas un `package-info.java` en el sub-package con la anotación `@NamedInterface`:

```
// En com/example/shop/orders/api/package-info.java
@org.springframework.modulith.NamedInterface("api")
package com.example.shop.orders.api;

// En com/example/shop/orders/events/package-info.java
@org.springframework.modulith.NamedInterface("events")
package com.example.shop.orders.events;

// En com/example/shop/orders/spi/package-info.java
@org.springframework.modulith.NamedInterface("spi")
package com.example.shop.orders.spi;
```

Ahora otros módulos pueden acceder a clases en `orders.api`, `orders.events`, y `orders.spi`, pero no a `orders.internal` o `orders.infrastructure`.

Control granular de dependencias: Puedes declarar que un módulo depende de una named interface específica, no de todo el módulo:

```
// En com/example/shop/shipping/package-info.java
@ApplicationModule(
    allowedDependencies = {"orders :: events"}
)
package com.example.shop.shipping;
```

Esto significa que "shipping" solo puede acceder al sub-package `orders.events`, no a `orders.api` ni a `orders.spi`. Esto es extremadamente poderoso para controlar el acoplamiento: shipping solo reacciona a eventos de orders, nunca llama directamente a la API de orders.

SPI (Service Provider Interface) como patrón: Un SPI permite que un módulo defina una interfaz que otro módulo implementa. El módulo de orders define `OrderFulfillmentStrategy` en `orders.spi`, y el módulo de shipping proporciona una implementación. Esto invierte la dependencia: orders no depende de shipping, sino que shipping implementa un contrato definido por orders.

Follow-ups anticipados:

¿Cuál es la diferencia entre una named interface y simplemente hacer un package público? — Sin `@NamedInterface`, todo sub-package es interno por convención de Spring Modulith. La anotación es la declaración explícita de "este sub-package es parte de mi API pública". Además, el nombre de la interfaz permite referenciarlo en `allowedDependencies`, habilitando control granular.

6. How does `ApplicationModules.verify()` enforce architecture at build time?

`verify()` es la pieza central del enforcement arquitectónico en Spring Modulith. No es un linter sugerente — es una verificación que falla el build si se violan las reglas.

Qué verifica exactamente:

La primera verificación es dependencias ilegales: si un módulo A importa una clase interna (sub-package) del módulo B, `verify()` falla con un mensaje claro indicando qué clase en qué módulo

violó qué frontera.

La segunda verificación es dependencias no declaradas: si usas `allowedDependencies` en `@ApplicationModule`, cualquier dependencia a un módulo no listado es una violación.

La tercera verificación es dependencias cíclicas: si el módulo A depende de B y B depende de A (directa o transitivamente), `verify()` falla. Los ciclos son el cáncer de la modularidad — hacen imposible entender, testear, o extraer módulos independientemente.

Cómo funciona internamente: `ApplicationModules.of()` analiza el classpath, construye un grafo de dependencias entre módulos basándose en las importaciones de Java (analiza el bytecode o el source, dependiendo de la configuración), y verifica las reglas contra ese grafo. No usa reflection en runtime — es análisis estático.

Integración en CI:

```
@Test
void moduleStructureIsValid() {
    ApplicationModules modules = ApplicationModules.of(ShopApplication.class);
    modules.verify();
}
```

Este test se ejecuta en cada build. Si un PR introduce una violación, el pipeline falla antes del merge. No hay forma de "saltárselo" sin modificar intencionalmente la definición del módulo — y eso es reviewable en el PR.

La diferencia con otros approaches: En un monolito sin Modulith, las fronteras son "reglas del equipo" que dependen de code review. En la práctica, bajo presión de deadlines, un desarrollador hace un import directo "temporal" que nunca se limpia. Con `verify()`, eso es literalmente imposible — el CI lo rechaza automáticamente.

Follow-ups anticipados:

¿No hace el build más lento? — El análisis es rápido (segundos, no minutos) porque opera sobre la estructura de packages y las importaciones, no sobre el código completo. El overhead es negligible comparado con el tiempo que ahorras evitando deuda técnica arquitectónica.

¿Puedo ignorar una violación temporalmente? — Sí, con

`@ApplicationModule(allowedDependencies = ...)` puedes declarar explícitamente una dependencia que normalmente sería ilegal. Pero eso queda documentado en el código y es visible en reviews. No hay "ignorar silenciosamente".

7. @ApplicationModuleTest — testing a single module in isolation

`@ApplicationModuleTest` es la anotación de Spring Modulith que bootstrapea un contexto de Spring reducido, conteniendo solo los beans del módulo bajo test y sus dependencias directas.

El problema que resuelve: En una aplicación Spring Boot grande, `@SpringBootTest` carga el `ApplicationContext` completo — todos los módulos, todas las configuraciones, todos los beans. Esto hace que los tests de integración sean lentos (30-60 segundos de startup) y frágiles (un cambio en el

módulo de shipping puede romper un test del módulo de orders porque comparten el mismo contexto).

Cómo funciona:

```
@ApplicationModuleTest
class OrderModuleIntegrationTests {

    @Autowired
    OrderService orderService;

    @Test
    void creatingOrderPublishesEvent(Scenario scenario) {
        scenario.stimulate(() -> orderService.createOrder(new
CreateOrderCommand(...)))
            .waitForEventOfType(OrderCreatedEvent.class)
            .toArriveAndVerify(event -> {
                assertThat(event.orderId()).isNotNull();
                assertThat(event.total()).isEqualTo(Money.of(99.99));
            });
    }
}
```

`@ApplicationModuleTest` solo bootstrapea el módulo "orders" y los módulos de los que "orders" depende explícitamente. Si "orders" depende de "catalog" (para resolver productos), se incluye "catalog". Pero "shipping", "payments", y otros módulos no se cargan.

Modos de bootstrapping:

`@ApplicationModuleTest(mode = BootstrapMode.STANDALONE)` carga solo el módulo, sin dependencias. Útil para tests unitarios de integración donde mockeas las dependencias externas.

`@ApplicationModuleTest(mode = BootstrapMode.DIRECT_DEPENDENCIES)` carga el módulo y sus dependencias directas. Es el default y el más útil para integration tests.

`@ApplicationModuleTest(mode = BootstrapMode.ALL_DEPENDENCIES)` carga el módulo y todas sus dependencias transitivas. Más cercano a un test end-to-end pero sin los módulos que no están en el path de dependencias.

Scenario API: La API `Scenario` permite testear flujos basados en eventos de forma declarativa: "estimula esta acción, espera este evento, verifica estas condiciones". Esto es mucho más limpio que publicar un evento manualmente y hacer `Thread.sleep()` esperando que el handler lo procese.

Follow-ups anticipados:

¿Es compatible con Testcontainers? — Sí. Puedes usar `@Testcontainers` junto con `@ApplicationModuleTest` para levantar un PostgreSQL en Docker para los tests del módulo. Cada módulo puede tener su propia configuración de Testcontainers.

¿Cómo manejo dependencias que no quiero cargar? — Usa `@MockBean` para reemplazar un bean de otro módulo con un mock. Esto es el equivalente a mockear un servicio externo en microservicios.

8. Detecting and preventing cyclic dependencies between modules

Las dependencias cíclicas son el problema arquitectónico más insidioso en cualquier sistema modular. Si A depende de B y B depende de A, no puedes entender, testear, o modificar uno sin el otro — efectivamente son un solo módulo con la ilusión de separación.

Detección automática: `ApplicationModules.verify()` detecta ciclos y falla con un mensaje que muestra el ciclo completo ($A \rightarrow B \rightarrow C \rightarrow A$). No necesitas herramientas adicionales — es built-in.

Estrategias para romper ciclos:

La técnica más poderosa es inversión de dependencia con eventos. Si Orders depende de Inventory para verificar stock, e Inventory depende de Orders para saber qué reservar, hay un ciclo. La solución: Orders publica un `OrderCreatedEvent`, e Inventory lo escucha y hace la reservación. Ahora Orders no conoce a Inventory — solo publica un evento. La dependencia es unidireccional: Inventory depende de Orders (para escuchar el evento), pero Orders no depende de Inventory.

```
// En el módulo Orders
@Service
public class OrderService {
    private final ApplicationEventPublisher events;

    public Order createOrder(CreateOrderCommand cmd) {
        Order order = Order.create(cmd);
        orderRepository.save(order);
        events.publishEvent(new OrderCreatedEvent(order.getId(),
order.getItems()));
        return order;
    }
}

// En el módulo Inventory (escucha el evento de Orders)
@Component
public class InventoryReservationListener {
    @ApplicationModuleListener
    void on(OrderCreatedEvent event) {
        inventoryService.reserveItems(event.items());
    }
}
```

La segunda técnica es crear un módulo de abstracción compartida. Si A y B dependen mutuamente, extraes la interfaz compartida a un módulo C (por ejemplo, `shared-kernel`). Tanto A como B dependen de C, pero no entre sí. C contiene interfaces, value objects, y eventos compartidos — nunca implementaciones.

La tercera técnica es mediation: introduces un módulo mediador que coordina entre A y B. En lugar de que A llame a B directamente, A publica un comando/evento, el mediador lo recibe, y orquesta la interacción con B.

Follow-ups anticipados:

¿Los ciclos transitivos también se detectan? — Sí. Si $A \rightarrow B \rightarrow C \rightarrow A$, `verify()` lo detecta. Los ciclos transitivos son más difíciles de identificar manualmente porque involucran cadenas largas, lo que hace la detección automática aún más valiosa.

¿Qué hago si el ciclo es "lógicamente necesario"? — En mi experiencia, cada vez que alguien dice que un ciclo es necesario, lo que realmente falta es un evento de dominio o una interfaz compartida. Si dos módulos son tan interdependientes que no puedes romper el ciclo, probablemente deberían ser un solo módulo.

9. Inter-module communication: direct calls vs domain events

Esta es una de las decisiones de diseño más importantes dentro de un modulith. Define el grado de acoplamiento entre módulos y tiene implicaciones directas en testabilidad, evolución, y eventual extracción a microservicios.

Direct calls (invocación síncrona): Un módulo llama directamente a un servicio de otro módulo a través de su API pública. Es simple, predecible, y transaccional (ambas operaciones ocurren en la misma transacción si compartes la misma `@Transactional boundary`).

```
// Módulo Orders llama directamente al módulo Catalog
@Service
public class OrderService {
    private final CatalogService catalogService; // Del módulo Catalog

    public Order createOrder(CreateOrderCommand cmd) {
        ProductInfo product = catalogService.getProduct(cmd.productId()); //
Llamada directa
        Order order = Order.create(cmd, product.price());
        return orderRepository.save(order);
    }
}
```

La ventaja es la simplicidad: el flujo es un call stack normal, el debugging es trivial, y si algo falla, la transacción completa hace rollback. La desventaja es que Orders tiene una dependencia directa en Catalog — si Catalog cambia su API, Orders se rompe.

Domain events (comunicación asíncrona): Un módulo publica un evento de dominio, y otros módulos lo escuchan y reaccionan. El publicador no sabe quién (ni cuántos) escuchan. Esto desacopla los módulos y facilita agregar nuevas reacciones sin modificar el módulo que publica.

```
// Módulo Orders publica un evento
events.publishEvent(new OrderPlacedEvent(order.getId(), order.getTotal()));

// Módulo Payments escucha y reacciona
@ApplicationModuleListener
void on(OrderPlacedEvent event) {
    paymentService.initiatePayment(event.orderId(), event.total());
}

// Módulo Notifications también escucha
@ApplicationModuleListener
void on(OrderPlacedEvent event) {
    notificationService.sendOrderConfirmation(event.orderId());
}
```

Mi regla de decisión: Uso direct calls para queries (obtener datos de otro módulo) y domain events para commands/reacciones (notificar que algo ocurrió). La razón es que los queries necesitan una respuesta inmediata (necesito el precio del producto AHORA para crear el pedido), mientras que las

reacciones pueden ocurrir de forma asíncrona y desacoplada (enviar un email de confirmación no necesita bloquear la creación del pedido).

Otra perspectiva: si la eliminación del módulo B no debería impedir que el módulo A funcione (degradadamente), entonces B debería reaccionar a un evento de A, no ser llamado directamente. Si A no puede funcionar sin B (necesita datos de B para operar), entonces una direct call es apropiada.

Follow-ups anticipados:

¿Los eventos son síncronos o asíncronos en Spring Modulith? — Por default, los `ApplicationEvent` de Spring son síncronos (ejecutados en el mismo thread y transacción). Spring Modulith con `@ApplicationModuleListener` los hace asíncronos y transaccionales: el evento se procesa en un thread separado, después de que la transacción del publicador haya committed. Esto es clave porque si el handler falla, no hace rollback la transacción original.

¿Qué pasa si el handler falla? — El Event Publication Registry (pregunta 11) se encarga de reintentar la entrega. El evento queda marcado como "incomplete" en la base de datos y un poller lo reintenta.

10. @ApplicationModuleListener vs @EventListener — the real difference

Ambas anotaciones permiten escuchar eventos de Spring, pero sus semánticas transaccionales y de entrega son fundamentalmente diferentes.

@EventListener (Spring estándar): El listener se ejecuta sincrónicamente, en el mismo thread y la misma transacción que el publicador. Si el listener lanza una excepción, la transacción completa (incluyendo la del publicador) hace rollback. Esto puede ser deseable (quiero que todo falle atómicamente) o catastrófico (un error al enviar un email de confirmación revierte la creación del pedido).

```
@EventListener
void on(OrderPlacedEvent event) {
    emailService.sendConfirmation(event); // Si esto falla, ¡el pedido NO se crea!
}
```

@TransactionalEventListener (Spring estándar): El listener se ejecuta después de que la transacción del publicador haya committed (por default, `phase = AFTER_COMMIT`). Si el listener falla, la transacción original ya se committed y no se revierte. El problema es que si la aplicación se cae entre el commit de la transacción y la ejecución del listener, el evento se pierde — no hay persistencia.

@ApplicationModuleListener (Spring Modulith): Combina lo mejor de ambos mundos. Semánticamente equivale a `@TransactionalEventListener` (ejecuta después del commit), pero agrega persistencia a través del Event Publication Registry. Antes de que la transacción del publicador haga commit, el evento se persiste en la tabla `event_publication` de la base de datos como parte de la misma transacción. Después del commit, el listener procesa el evento. Si la aplicación se cae antes de que el listener termine, el evento queda marcado como "incomplete" en la base de datos y se reintenta al reiniciar.


```
@ApplicationModuleListener // Ejecuta después del commit, con entrega
garantizada
void on(OrderPlacedEvent event) {
    emailService.sendConfirmation(event);
    // Si falla, se reintenta automáticamente
    // Si la app se cae, se reintenta al reiniciar
}
```

Cuándo usar cada uno:

`@EventListener` solo para operaciones que DEBEN ser atómicas con la transacción original (validaciones que deben revertir todo si fallan). Esto es raro en comunicación inter-módulo.

`@ApplicationModuleListener` para toda comunicación entre módulos basada en eventos. Es el default correcto.

`@TransactionalEventListener` prácticamente nunca en un modolith que usa Spring Modulith, porque `@ApplicationModuleListener` es estrictamente superior (agrega persistencia sin costo adicional de configuración).

Follow-ups anticipados:

¿Qué pasa con el ordering de listeners? — Si múltiples listeners escuchan el mismo evento, el orden no está garantizado con `@ApplicationModuleListener` (porque son asíncronos). Si necesitas orden, los listeners deben ser independientes entre sí, o debes usar un orchestrator que coordine la secuencia.

11. The Event Publication Registry — reliable events without Kafka

El Event Publication Registry es posiblemente la feature más valiosa de Spring Modulith para sistemas que necesitan eventos confiables sin la complejidad operacional de un message broker externo.

El problema que resuelve: Los eventos de Spring (`ApplicationEvent`) son in-memory. Si la aplicación se reinicia, se cae, o el listener falla, el evento se pierde. La solución tradicional es agregar Kafka o RabbitMQ, lo que introduce un componente de infraestructura adicional que necesita deployment, monitoreo, y expertise para operar. Para un modolith donde los eventos son internos (no cruzan boundaries de red), un message broker es overkill.

Cómo funciona: El Registry intercepta la publicación de eventos y, antes de que la transacción del publicador haga commit, persiste el evento en una tabla de la base de datos (`event_publication`). La tabla almacena el evento serializado (JSON), el tipo del listener target, y un status (`incomplete/completed`). Después del commit, el listener procesa el evento. Si el procesamiento es exitoso, el registro se marca como "completed". Si falla, queda como "incomplete" y se reintenta.

Configuración:

```
// application.yml
spring:
  modulith:
```

```
events:
  jdbc:
    schema-initialization:
      enabled: true # Crea la tabla automáticamente
    republish-outstanding-events-on-restart: true
```

La tabla creada se ve conceptualmente así:

```
CREATE TABLE event_publication (
  id UUID PRIMARY KEY,
  listener_id VARCHAR(512),      -- Identificador del listener target
  event_type VARCHAR(512),      -- Clase del evento
  serialized_event TEXT,        -- JSON del evento
  publication_date TIMESTAMP,
  completion_date TIMESTAMP     -- NULL si incompleto
);
```

Reintento y cleanup: Los eventos incompletos se reintentan al reiniciar la aplicación (si `republish-outstanding-events-on-restart` está habilitado). También puedes configurar un `IncompleteEventPublications` para reintentar periódicamente sin reiniciar. Los eventos completados se limpian con `CompletedEventPublications.deleteCompletedPublicationsBefore(Duration.ofDays(1))`, típicamente ejecutado como un `scheduled job`.

Follow-ups anticipados:

¿Cuándo SÍ necesito Kafka entonces? — Cuando los eventos cruzan `boundaries` de proceso (microservicios), cuando necesitas `replay` de eventos históricos (`event sourcing`), cuando múltiples aplicaciones independientes necesitan consumir los mismos eventos, o cuando el volumen de eventos es tan alto que la base de datos relacional no es eficiente como `event store`. En un `modulith`, el `Event Publication Registry` cubre el 90% de los casos.

¿Qué pasa si el evento se procesa dos veces (at-least-once)? — Los `listeners` deben ser `idempotentes`. Si el `listener` envía un email, debe verificar si ya se envió antes de enviarlo de nuevo (usando una tabla de deduplicación o el ID del evento como `idempotency key`).

¿Es escalable? — Para `moduliths` con volúmenes moderados de eventos (miles por minuto), la tabla relacional es más que suficiente. Si tienes millones de eventos por minuto, probablemente ya no es un `modulith` sino que necesitas una arquitectura distribuida con Kafka.

12. Handling transactions across modules without distributed sagas

En un `modulith`, la gran ventaja es que todos los módulos comparten el mismo proceso y, potencialmente, la misma base de datos. Esto permite transacciones `ACID` locales que en microservicios requerirían sagas complejas.

Transacción compartida (el caso simple): Si ambos módulos usan la misma base de datos, una `@Transactional` en el servicio que orquesta la operación envuelve todo en una transacción `ACID`. Si el pago falla después de crear el pedido, todo hace `rollback`.

```
@Service
```

```
@Transactional
public class CheckoutService {
    // Ambos en la misma transacción ACID
    public void checkout(CheckoutCommand cmd) {
        Order order = orderService.createOrder(cmd);           // Módulo Orders
        paymentService.processPayment(order.getTotal());        // Módulo Payments
        // Si processPayment() falla, createOrder() se revierte automáticamente
    }
}
```

Esto es posible SOLO porque ambos módulos comparten el mismo datasource. En microservicios, esto sería imposible — necesitarías una saga.

Eventos con transaccionalidad: Cuando no quieres acoplamiento directo pero sí necesitas consistencia eventual, usas el patrón: "haz tu operación local + publica un evento en la misma transacción (gracias al Event Publication Registry), y el handler del evento hará su parte en su propia transacción".

```
@Transactional
public Order createOrder(CreateOrderCommand cmd) {
    Order order = orderRepository.save(Order.create(cmd));
    events.publishEvent(new OrderCreatedEvent(order.getId()));
    return order;
    // Al hacer commit: (1) el order se persiste, (2) el evento se persiste en
    event_publication
    // Después del commit: el listener de Payments procesa el pago en su propia
    transacción
}
```

Si el pago falla, el pedido ya existe. El listener se reintenta, y si continúa fallando, puedes implementar compensación: publicar un `PaymentFailedEvent` que el módulo de Orders escucha para cancelar el pedido.

El pattern de Reservation: Para operaciones donde necesitas "reservar" algo de forma provisional, usas un estado intermedio. El pedido se crea en estado **PENDING**, el pago se intenta, y si es exitoso, el pedido pasa a **CONFIRMED**. Si falla, pasa a **CANCELLED**. No hay distribución de transacciones — cada paso es una transacción local con estados de máquina de estados bien definidos.

Follow-ups anticipados:

¿Cuándo necesitaría distributed transactions en un modolith? — Casi nunca. Si usas una sola base de datos, las transacciones ACID locales cubren todo. Si usas database-per-module (pregunta 13), podrías necesitar el patrón de outbox + eventual consistency, pero no distributed transactions (2PC), que son frágiles y lentas.

13. Database-per-module vs shared schema inside a modolith

Esta decisión afecta directamente cuánto aislamiento tienen los módulos y cuán fácil será extraerlos como microservicios en el futuro.

Shared schema (el default pragmático): Todos los módulos usan la misma base de datos y el mismo schema. La ventaja es la simplicidad operacional (una conexión, un backup, un monitoreo) y la posibilidad de transacciones ACID cross-module. La restricción que debes imponer es disciplina

de ownership: cada tabla pertenece a un módulo, y otros módulos no hacen queries directos contra tablas ajenas — siempre pasan por la API del módulo owner. Esto no se puede enforcer automáticamente con Spring Modulith (no analiza SQL), pero puedes validarlo con naming conventions (prefijo de módulo en las tablas: `orders_orders`, `payments_transactions`) y code review.

Schema-per-module: Cada módulo tiene su propio schema de PostgreSQL. Esto proporciona un aislamiento fuerte — un módulo no puede "accidentalmente" hacer un JOIN contra tablas de otro módulo porque no tiene acceso al schema ajeno. Las migraciones son independientes por módulo. Es un excelente punto intermedio antes de microservicios.

```
# Configuración de múltiples datasources por módulo
spring:
  datasource:
    orders:
      url: jdbc:postgresql://localhost:5432/shop?currentSchema=orders
    payments:
      url: jdbc:postgresql://localhost:5432/shop?currentSchema=payments
```

La desventaja es que pierdes transacciones ACID cross-schema (cada módulo tiene su propio DataSource y TransactionManager), y la configuración de Spring para múltiples datasources es verbosa.

Database-per-module: Cada módulo tiene su propia base de datos física. Es el aislamiento máximo y es equivalente a lo que tendrías en microservicios. Rara vez es necesario en un modulith a menos que un módulo tenga requerimientos de storage radicalmente diferentes (por ejemplo, el módulo de search usa Elasticsearch mientras el resto usa PostgreSQL).

Mi recomendación: Shared schema con ownership disciplinada para empezar. Cuando un módulo sea candidato para extracción a microservicio, migra primero a schema-per-module dentro del modulith (que es un cambio mucho más simple que la extracción completa), valida que todo funciona con eventual consistency, y solo entonces extrae.

Follow-ups anticipados:

¿Cómo hago reporting o analytics cross-module si no puedo hacer JOINS? — Crea una vista OLAP (materialized view o data warehouse) que consume eventos de todos los módulos y construye las vistas cross-domain que reporting necesita. Esto es exactamente lo que harías con microservicios, y es una buena práctica incluso con shared schema.

14. Auto-generating C4 diagrams and Application Module Canvas

Spring Modulith puede generar documentación visual a partir del código, lo que resuelve el eterno problema de "los diagramas no reflejan la realidad".

Application Module Canvas: Es una representación tabular de cada módulo que muestra sus beans publicados (API), sus dependencias, los eventos que publica, los eventos que escucha, y las propiedades de configuración que consume. Se genera con una línea:

```
@Test
```

```

void generateDocumentation() {
    ApplicationModules modules = ApplicationModules.of(ShopApplication.class);
    new Documenter(modules)
        .writeModuleCanvases()           // Genera canvas por módulo
        .writeModulesAsPlantUml()        // Genera diagramas C4 en PlantUML
        .writeIndividualModulesAsPlantUml(); // Un diagrama por módulo
}

```

Esto genera archivos en `target/modulith-docs/` que puedes renderizar con cualquier herramienta PlantUML (o integrar en tu pipeline de documentación).

Diagramas C4 generados: El nivel 2 (Container diagram) muestra los módulos y sus relaciones — qué módulo depende de cuál, y si la dependencia es por invocación directa o por eventos. Esto se genera directamente del análisis de código, así que siempre está actualizado.

Integración en CI/CD: Genero los diagramas como parte del build y los publico en una wiki o un sitio estático (GitHub Pages, Backstage). Cada merge a main regenera la documentación automáticamente. Esto elimina el trabajo manual de actualizar diagramas — el código IS la documentación.

Follow-ups anticipados:

¿Reemplaza esto a diagramas manuales? — Para los niveles 2 y 3 de C4 (containers y components), sí. Para el nivel 1 (System Context, que muestra el sistema completo con actores y sistemas externos), sigue siendo necesario un diagrama manual porque Spring Modulith solo ve la estructura interna de la aplicación.

15. Observability in a modulith with Micrometer and Actuator

En un modulith, la observabilidad tiene una ventaja sobre microservicios: no hay network hops entre módulos, así que el distributed tracing es más simple. Pero sigue siendo necesario entender qué módulo está consumiendo recursos y dónde están los cuellos de botella.

Micrometer para métricas por módulo: Cada módulo puede exponer métricas custom vía Micrometer, y todas se agregan en el mismo endpoint `/actuator/metrics`. Para diferenciar métricas por módulo, uso tags:

```

@Service
public class OrderService {
    private final MeterRegistry registry;
    private final Counter ordersCreated;

    public OrderService(MeterRegistry registry) {
        this.ordersCreated = Counter.builder("orders.created")
            .tag("module", "orders")
            .register(registry);
    }

    public Order createOrder(CreateOrderCommand cmd) {
        Order order = // ... crear orden
        ordersCreated.increment();
        return order;
    }
}

```

Spring Modulith Observability: Spring Modulith integra con Micrometer Observation API para auto-instrumentar las invocaciones entre módulos. Cuando un módulo llama a otro, automáticamente se crea un span con el nombre del módulo origen, el módulo destino, y la duración. Esto te da un trace de las interacciones inter-módulo sin escribir código de instrumentación.

```
spring:
  modulith:
    observability:
      enabled: true
```

Actuator para health checks por módulo: Cada módulo puede contribuir su propio health indicator. Si el módulo de pagos depende de un gateway externo, su health check verifica la conectividad con ese gateway. El endpoint `/actuator/health` agrega todos los indicadores.

Follow-ups anticipados:

¿Cómo diferencio problemas de rendimiento entre módulos? — Los traces generados por Spring Modulith muestran claramente cuánto tiempo se consume en cada módulo. Si el endpoint `/api/checkout` tarda 2 segundos, el trace muestra: OrderModule (50ms) → CatalogModule (100ms) → PaymentModule (1800ms). El bottleneck está claramente en Payments.

16. Migrating a legacy monolith → modulith (the strangler-fig, indoors)

La metáfora "strangler fig indoors" es brillante: aplicas el patrón Strangler Fig Pattern, pero en lugar de extraer a microservicios externos, "estranguila" el código legacy con módulos internos dentro del mismo deployment.

Paso 1 — Assessment: Agrega Spring Modulith al proyecto existente sin cambiar nada de código. Ejecuta `ApplicationModules.of(App.class)` y observa qué módulos detecta y qué violaciones encuentra. Esto te da un mapa de la situación actual: qué dependencias existen, dónde hay ciclos, y cuán acoplado está todo.

Paso 2 — Reorganización de packages: Mueve clases de la estructura "por capa" (controllers/, services/, repositories/) a la estructura "por dominio" (orders/, payments/, catalog/). Este es el refactoring más mecánico y el más impactante. Tu IDE puede hacer la mayoría del trabajo con "Move Class". No cambias comportamiento — solo ubicación de archivos.

Paso 3 — Identificar las dependencias más dañinas: El output de `verify()` lista todas las violaciones. Ordena por frecuencia y severidad. Un ciclo entre dos módulos core es más urgente que una dependencia a una utility interna. Aborda los ciclos primero usando la técnica de inversión con eventos (pregunta 8).

Paso 4 — Cortar dependencias incrementalmente: No intentes arreglar todo de una vez. Introduce `@ApplicationModule(allowedDependencies = ...)` gradualmente, empezando por los módulos más estables (los que cambian menos). Cada PR corta una o dos dependencias ilegales. Con el tiempo, `verify()` pasa limpio.

Paso 5 — Introducir eventos de dominio: Reemplaza las llamadas directas entre módulos que representan side-effects (notificaciones, auditoría, sincronización) con eventos de dominio. Mantén las llamadas directas solo para queries que necesitan respuesta inmediata.

Paso 6 — Separar schemas (opcional): Una vez que las dependencias están limpias, puedes mover tablas a schemas separados para enforced el ownership de datos a nivel de base de datos.

Follow-ups anticipados:

¿Cuánto tiempo toma la migración? — Depende del tamaño, pero para un monolito de 100-200K LOC, el assessment toma 1-2 días, la reorganización de packages 2-4 semanas, y la limpieza de dependencias es un esfuerzo continuo de 2-6 meses en paralelo con feature work. No es un proyecto separado — se integra en el trabajo diario.

¿Cómo convenzo al negocio? — No pides permiso para "refactorizar la arquitectura". Cada ticket de feature que toques, dejas el código más modular de lo que lo encontraste (Boy Scout Rule). La reorganización de packages no cambia comportamiento y se puede hacer como parte de cualquier PR.

17. Extracting a module into a microservice when scale actually demands it

La extracción de un módulo a microservicio es una operación quirúrgica que debe hacerse por razones concretas, no por moda arquitectónica.

Pre-requisitos para una extracción limpia: El módulo ya tiene fronteras claras (pasa `verify()` sin violaciones). La comunicación con otros módulos es principalmente vía eventos de dominio, no llamadas directas. El módulo tiene su propio schema o al menos ownership claro de sus tablas. Los tests del módulo pasan en modo STANDALONE.

Proceso de extracción:

Primero, cambias la comunicación inter-módulo de eventos in-process a eventos sobre un message broker. Reemplazas `@ApplicationModuleListener` con un consumidor de Kafka/SQS. El publicador cambia de `ApplicationEventPublisher` a un `KafkaTemplate`. Los eventos mantienen la misma estructura (el DTO del evento se extrae a un módulo compartido o a un schema registry).

Segundo, si el módulo compartía schema con otros, migras sus tablas a una base de datos independiente. Las migraciones del módulo se ejecutan contra su nueva base de datos.

Tercero, reemplazas las llamadas directas (imports de Java) con llamadas HTTP/gRPC. La interfaz del servicio del módulo se convierte en un API REST o gRPC, y los consumidores usan un client (Feign, WebClient, gRPC stub) en lugar del import directo.

Cuarto, deployas el módulo como un artefacto separado con su propio pipeline de CI/CD, su propio Dockerfile, y su propia configuración de Kubernetes.

La ventaja del modulith como paso intermedio: Cada uno de estos pasos se puede hacer incrementalmente y testear en producción antes de avanzar al siguiente. Si la extracción causa

problemas, puedes revertir a la versión modolith rápidamente. Esto es fundamentalmente más seguro que diseñar microservicios desde cero.

Follow-ups anticipados:

¿Cuánto overhead agrega la extracción? — Significativo. Ahora tienes latencia de red (1-10ms por llamada), necesidad de circuit breakers, retry logic, contract testing, un pipeline de CI/CD adicional, monitoring independiente, y on-call para el servicio. Ese costo debe estar justificado por un beneficio concreto.

18. Concrete signals it's time to split a module out (and signals it isn't)

Señales de que SÍ es momento:

El módulo necesita escalar independientemente porque recibe 100x más carga que el resto del sistema. Por ejemplo, el módulo de search recibe millones de queries mientras el módulo de pedidos recibe miles. Escalar todo el modolith para satisfacer a search es un desperdicio.

El módulo tiene requerimientos de disponibilidad diferentes. Si el módulo de pagos necesita 99.99% uptime pero el módulo de reportes puede tolerar 99.9%, acoplarlos en el mismo proceso significa que un bug en reportes puede derribar pagos.

Equipos diferentes con cadencias de release diferentes necesitan deployar el módulo independientemente. Si el equipo de pagos quiere deployar 10 veces al día pero el modolith se deploya una vez por semana, hay fricción organizacional real.

El módulo necesita un stack tecnológico diferente. Si el módulo de ML necesita Python con TensorFlow y el resto es Java, no pueden coexistir en el mismo proceso.

Señales de que NO es momento:

"Los microservicios son las mejores prácticas" — esto no es una señal, es una falacia de autoridad. Las mejores prácticas dependen del contexto.

"Queremos estar preparados para escalar" — la preparación correcta es un modolith bien diseñado, no microservicios prematuros. Puedes escalar un modolith horizontalmente (múltiples instancias detrás de un load balancer) hasta un punto sorprendentemente alto.

"El equipo quiere aprender Kubernetes" — una motivación válida para un proyecto de aprendizaje, no para decisiones arquitectónicas de producción.

El módulo tiene alta cohesión con otros módulos — si la mayoría de las operaciones del módulo requieren datos de otros módulos (muchos JOINS, muchas llamadas síncronas), la extracción convertirá esos JOINS baratos en llamadas de red caras.

Follow-ups anticipados:

¿Cómo cuantifico si el beneficio justifica el costo? — Calcula el "distributed systems tax": cuántas horas-ingeniero por mes gastarás en infraestructura de microservicios (service mesh, tracing, contract testing, on-call adicional). Si ese costo supera el beneficio de la escala o deployment independiente, no vale la pena.

19. Organizing teams around modules instead of services — Conway's Law revisited

La Ley de Conway dice que las organizaciones diseñan sistemas que reflejan su estructura de comunicación. El "Inverse Conway Maneuver" propone diseñar la estructura de equipos para producir la arquitectura que desees.

Módulos como unidades de ownership: En un modulith, cada módulo tiene un equipo owner (o un sub-equipo, o incluso un par de personas). El owner tiene autoridad sobre la API pública del módulo, las decisiones de diseño interno, y la priorización de deuda técnica dentro del módulo. Esto proporciona autonomía sin el costo de microservicios.

La ventaja sobre "equipos por servicio": En microservicios, cada equipo tiene su propio servicio, su propio pipeline, su propio monitoreo, y su propia base de datos. Esto crea overhead: 10 equipos con 10 servicios son 10 pipelines de CI/CD, 10 configuraciones de monitoring, 10 rotaciones de on-call. En un modulith, compartes la infraestructura (un pipeline, un deployment, un sistema de monitoring) pero mantienes la autonomía de dominio. Un equipo puede refactorizar completamente los internals de su módulo sin coordinar con nadie, siempre que la API pública no cambie.

Inner source model: Cualquier ingeniero puede proponer cambios a cualquier módulo (como en open source), pero el owner hace el code review y tiene la última palabra. Esto previene bloqueos ("necesito un cambio en el módulo de pagos pero el equipo está ocupado") sin perder la coherencia arquitectónica del módulo.

Cómo escala: Un equipo puede ownar múltiples módulos (típico en equipos pequeños). Un módulo nunca debería tener múltiples equipos como owners (eso diluye la responsabilidad). Cuando un módulo crece demasiado para un equipo, se subdivide en sub-módulos o se extrae como un servicio independiente con su propio equipo.

Follow-ups anticipados:

¿Cómo manejas dependencias cross-module que requieren coordinación? — Los cambios en APIs públicas de un módulo requieren coordinación con los consumidores. Uso "expand-and-contract" (API evolution): agrego la nueva API, migro los consumidores, y depreco la vieja. Nunca rompo la API existente en un solo PR.

20. Defending "we don't need Kubernetes yet" in a design review

Esta pregunta es sobre la habilidad de defender una posición técnica con argumentos concretos, no dogma. La respuesta correcta no es "Kubernetes es malo", sino "Kubernetes no está justificado dado nuestro contexto actual".

El argumento central: Kubernetes resuelve tres problemas: orquestación de containers a escala, scheduling de workloads entre múltiples nodos, y auto-healing de procesos. Si tu sistema es un modulith con 2-5 instancias detrás de un ALB, no necesitas orquestación (ECS con Fargate o incluso EC2 con un Auto Scaling Group es suficiente), no necesitas scheduling complejo (cada

instancia es idéntica), y el auto-healing lo maneja el Auto Scaling Group (reemplaza instancias unhealthy) o ECS (reinicia containers fallidos).

Costo real de Kubernetes: El control plane de EKS cuesta ~\$73/mes por cluster, lo cual parece barato, pero el costo real está en el expertise necesario para operarlo: networking (VPC CNI, ingress controllers, service mesh), security (RBAC, pod security policies, image scanning), storage (persistent volumes, EBS CSI driver), monitoring (Prometheus operator, Grafana, alert manager), y el tiempo de ingeniería para mantener todo esto actualizado (upgrades de versiones de EKS cada 12-14 meses son no-triviales). Un equipo de 5-10 ingenieros dedicará entre 20-40% del tiempo de uno o dos personas a "alimentar" Kubernetes. Ese es el costo que nadie menciona en los design reviews.

Alternativas concretas:

Para un modolith con 2-10 instancias, ECS Fargate es significativamente más simple: defines un task definition (el equivalente a un pod spec), un service (que mantiene N instancias running), y un ALB. No hay nodos que gestionar, no hay networking de cluster que configurar, no hay versiones de Kubernetes que upgradear. El deployment se reduce a actualizar el task definition con la nueva imagen de Docker.

Para aún más simplicidad, AWS App Runner o Railway/Render proporcionan deployment de containers con zero infrastructure management. Pierdes flexibilidad pero ganas velocidad de desarrollo.

Cuándo SÍ necesitas Kubernetes: Cuando tienes 20+ microservicios con diferentes equipos que necesitan self-service de deployment. Cuando tienes workloads con requerimientos de scheduling complejos (GPU, node affinity, spot instances con fallback). Cuando necesitas service mesh (Istio/Linkerd) para mTLS y traffic management avanzado. Cuando tienes un equipo de plataforma dedicado a mantener el cluster.

Cómo presento el argumento en un design review: "Kubernetes agrega valor cuando la complejidad de orquestación justifica el overhead operacional. Con un modolith de N instancias, ECS Fargate nos da auto-scaling, health checks, rolling deployments, y service discovery por una fracción del costo operacional. Si en 12 meses tenemos 15+ servicios y un equipo de plataforma, revisamos la decisión. Aquí está el ADR con el análisis completo."

Follow-ups anticipados:

¿No estamos creando deuda técnica al no usar Kubernetes desde el inicio? — La deuda técnica sería adoptar prematuramente una herramienta que no necesitas y gastar meses aprendiéndola y operándola. Migrar de ECS a EKS cuando lo necesites es un proyecto acotado (2-4 semanas para un equipo experimentado). Migrar de Kubernetes a algo más simple cuando te das cuenta que no lo necesitabas es mucho más doloroso, porque ya construiste dependencias sobre features específicas de K8s (ConfigMaps, Secrets, CronJobs, RBAC).

¿Y si necesito Kubernetes para el CV? — Es una motivación válida para proyectos personales, no para decisiones de arquitectura de producción. El impacto en el equipo y el negocio es real, y la responsabilidad del arquitecto es tomar la decisión correcta para el contexto, no la que maximice el CV individual.